CoTune

Active Suspension & Rotary Inverted Pendulum

Submitted By:

Adham Ehab Saleh

Abdelrahman Hany Abdelrahman

Ahmed Mohamed Ramadan

Ahmed Yasser Hosny

Khalid Amin Abdullah

Omar Mohamed Fathy

Mechatronics Program

Graduation Project Report

Supervisor:

Dr. Mohamed Omar

28/1/2026

DECLARATION

We hereby certify that this Project submitted as part of our partial fulfilment of BSc in Mechatronics Engineering is entirely our own work, that we have exercised reasonable care to ensure its originality, and does not to the best of our knowledge breach any copyrighted materials, and have not been taken from the work of others and to the extent that such work has been cited and acknowledged within the text of our work.

Signed: by

Adham Ehab Saleh Mokhtar	
Abdelrahman Hany Abdelrahman Omar	
Ahmed Mohamed Ramadan	
Ahmed Yasser Hosny	
Khaled Amin	
Omar Mohamed Fathy Ibrahim	

Date: 28th of January 2026

ACKNOWLEDGMENT

We would like to express our sincere gratitude to our supervisor, **Dr. Mohamed Omar**, for his guidance and continuous support throughout this project.

Abstract

This thesis focuses on the development of new experimental platforms intended to extend the capabilities of the CoTune Control Laboratory, with the aim of enhancing hands-on learning in control systems education through real-hardware experimentation. The project introduces two laboratory-scale experiments—an active suspension system and a rotary inverted pendulum—designed to complement theoretical coursework with practical implementation and experimental validation.

Both experimental setups were developed from the ground up, starting with mechanical design and hardware integration, followed by system modeling and real-time experimental operation. The platforms are designed to support remote and flexible experimentation, allowing users to configure parameters and observe system behavior through live visual feedback, without requiring physical presence in the laboratory.

The active suspension experiment represents a quarter-car model and enables the study of vertical vehicle dynamics, disturbance effects, and suspension performance using a compact laboratory-scale setup. The rotary inverted pendulum experiment represents a classical unstable system and serves as a benchmark platform for studying nonlinear dynamics, instability, and real-time feedback control.

Together, these experimental platforms expand the range of control problems addressed within the CoTune Laboratory and provide a foundation for future integration into remote-access educational environments, supporting a closer connection between control theory and practical implementation.

Table of Contents

List of Figures

List of Tables

1. Introduction

1.1. Background

Traditional control laboratories are a core component of engineering education, and most engineering students gain some level of hands-on experience during their studies. However, these laboratories are typically constrained by physical presence, fixed schedules, limited hardware availability, and restricted experiment time. As a result, students are often unable to freely test control theories, explore parameter variations, or repeat experiments outside scheduled lab sessions.

In addition, increasing student numbers and limited laboratory resources make it difficult to provide equal experimental access to all students. These constraints reduce the effectiveness of laboratory-based learning, particularly in control engineering, where repeated experimentation and real-time observation are essential for developing intuition and practical understanding.

The primary limitation addressed in this work is not the absence of control laboratories, but rather the lack of remote, flexible, and continuously accessible experimental platforms that allow students to interact with real hardware without being physically present in the laboratory.

1.2. CoTune Laboratory Concept

The Cotune Control Laboratory was developed to overcome the limitations of conventional control labs by enabling remote experimentation on real physical systems. CoTune allows students to access laboratory experiments through an online platform, configure system and controller parameters, and observe real-time system behavior without requiring physical presence in the lab.

Instead of replacing traditional laboratories, CoTune extends their functionality by decoupling experimentation from location and time constraints. This approach allows students to test control theories, analyze system responses, and validate design choices using real hardware, while maintaining the realism and non-ideal behavior of physical systems.

The CoTune Laboratory is designed as a unified framework that integrated multiple experimental platforms, real-time control execution, live data acquisition, and visual feedback, providing consistent user experience across different experiments.

1.3. Remote Experimentation and Live Interaction Framework

A core feature of the Cotune Laboratory is the ability to perform experiments remotely without requiring physical presence in the lab. This capability allows students to interact with real experimental setups through an online platform, overcoming common limitations related to lab availability, scheduling, and physical access.

Using the CoTune web interface, students can select an experiment, set the relevant parameters, and run the experiment directly on the physical hardware. The system executes the experiment in real time, while measured signals and system responses are streamed back to the user. In parallel, a live camera feed provides continuous visual feedback, allowing students to observe the physical motion of the system as the experiment is running.

This interaction model preserves the essential characteristics of hands-on experimentation, including real-time dynamics and non-ideal behavior, while enabling experiments to be performed from any location. Screenshots of the web interface and live camera feed are included to illustrate the structure of the platform and the user interaction flow.

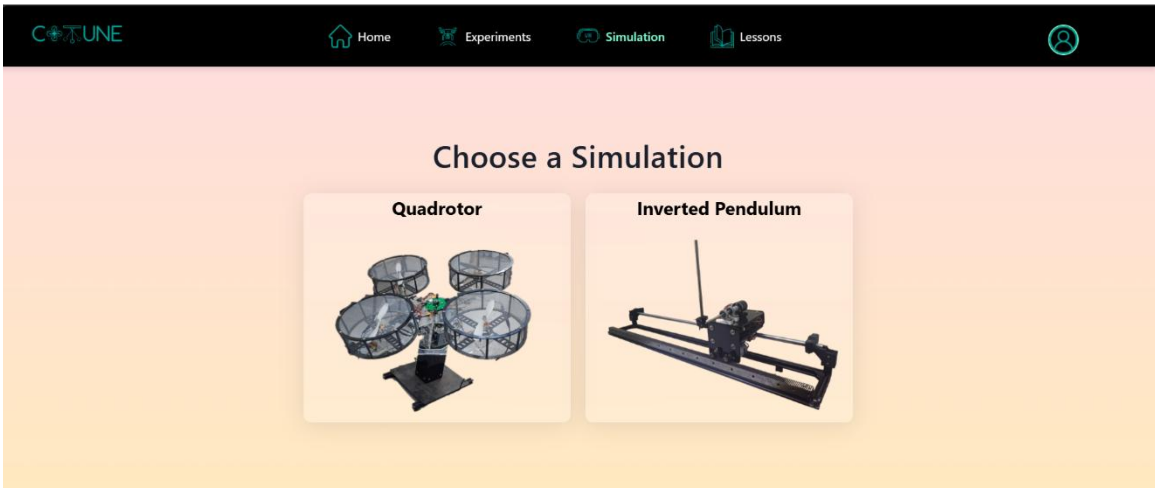


Figure 1.CoTune web Interface

1.4. Existing Experimental Platforms

Before the work presented in this thesis, the CoTune Laboratory already included several experimental platforms designed to cover a range of control applications. These platforms include:

- A quadrotor system.
- A linear inverted pendulum
- A ball balancing setup.
- A robotic arm.
- A three-dimensional scanning system.

Each of these experiments addresses different aspects of control systems, such as stability, multivariable control, motion control, and sensor-based feedback. Together, they form a solid foundation for practical control education within the CoTune framework. However, the existing platforms did not fully cover certain important control problems related to disturbance rejection and classical benchmark systems, motivating the addition of new experiments.

1.5. Motivation for the Newly Added Experiments

The selection of the active suspension and the rotary inverted pendulum was driven by the need to expand the range of control problems accessible through the CoTune Laboratory. The focus was not on adding more experiments for volume, but on introducing systems that offer clear educational value and complement the existing setups.

Relevance to vehicle dynamics and suitability for studying disturbance effects and performance trade-offs in controlled mechanical systems motivated the choice of the **active suspension system**. It allows students to directly observe how control actions influence system response under external excitations, making it well suited for both conceptual understanding and practical testing.

Considered a classical unstable system widely used in control education, the **rotary inverted pendulum** was selected for its ability to clearly demonstrate instability and real-time stabilization. Its dynamics offer an effective platform for testing control strategies under challenging conditions.

Together, these two experiments extend the laboratory's coverage while fitting naturally within the existing CoTune framework.

2. Active Suspension Experiment

2.1. Introduction

2.1.1. Background

The automotive industry is currently undergoing a significant transformation driven by advances in control systems, electronics and embedded software. Modern vehicles are no longer purely mechanical systems; instead, they are increasingly becoming more complex cyber-physical systems. This evolution has led to higher expectations from passengers, who now demand not only reliable transportation, but also enhanced ride comfort, safety, and overall driving experience.

Among the vehicle subsystems, the suspension system plays a crucial role in isolating the vehicle body from road disturbance while ensuring stable vehicle behavior. Traditional passive suspension systems are limited by fixed mechanical properties that prevent adaptation to varying road conditions. These limitations have motivated the development of advanced suspension technologies that integrate control algorithms with mechanical design.

2.1.2. Suspension System

The suspension system is a fundamental subsystem in any road vehicle, responsible for supporting the vehicle weight, isolating the vehicle body from road disturbances, and ensuring stable vehicle behavior during motion by maintaining the traction force between the tire and the road surface.

A typical suspension system consists of mechanical elements such as springs, dampers, and linkages. The springs support the vehicle load and absorb energy from road irregularities, while the dampers dissipate this energy to limit excessive oscillations. Together, these components transfer forces between the vehicle body and the road surface while reducing the unwanted vibrations.

Conventional suspension systems require a compromise between ride comfort and road handling. Improving one usually leads to reduction in the other, which limits the overall system performance.

2.1.3. Passive Suspension

Passive suspension is the traditional suspension configuration used in most conventional vehicles. The dynamic behavior of a passive suspension system is entirely determined by the fixed mechanical properties of these components.

Due to the fixed nature of passive suspension parameters, these systems are unable to adapt to changing road conditions or driving situations.

2.1.4. Active Suspension

The limitations of the passive suspension have motivated the development of active suspension technology. Addressing this limitation through the integration of mechanical components, sensors, actuators and control algorithms. This actuator can apply controlled forces between the vehicle body and the wheel assembly based on sensor measurements and control commands. Now the system can respond to road disturbance in real time rather than relying solely on passive energy dissipation.

2.1.5. Classification of Active Suspension

There are 2 main classes of active suspension

- Semi-Active Suspension

The active element is embedded within the damper to vary the damping coefficient. This is achieved by varying the diameter of an orifice or applying magnetic field to small magnetic particles suspended within the hydraulic fluid in the damper. They are still limited because they can only dissipate energy; they cannot generate an independent force to lift the body or push the wheel down

- Full-Active Suspension

The active element is separate and parallel to the damper to producing force. Such element can be a linear motor or a hydraulic cylinder.

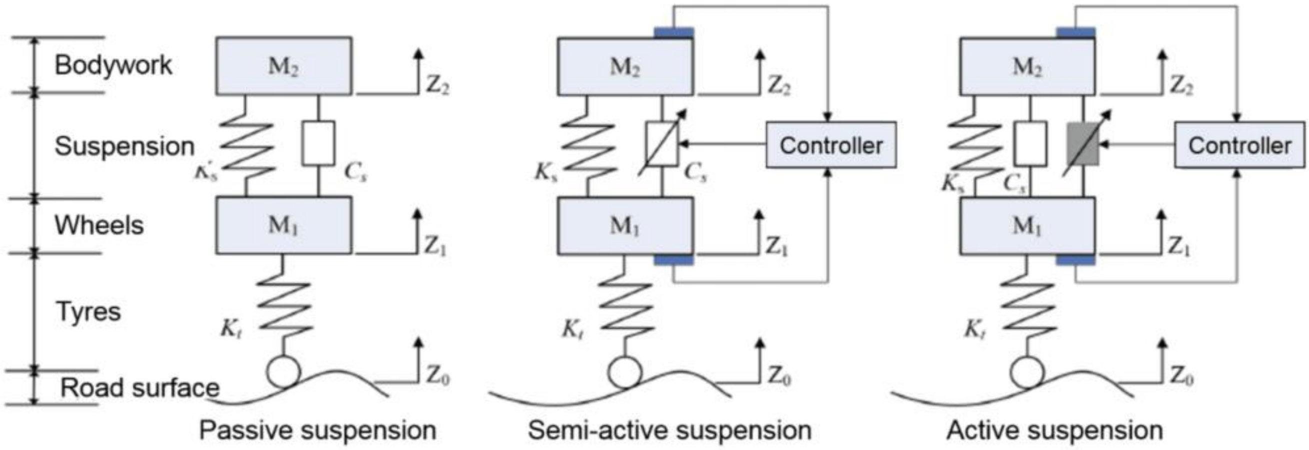


Figure 2. Suspension Types

2.1.6. Comparison between Active and Semi-Active Suspension

While Full-Active suspension offers superior performance, the high complexity and large power consumption are obstacles to their adoption into everyday vehicles keeping them as high-end feature offered only in luxury vehicles.

Semi-Active Suspension offers good compromise. Because it has better performance compared to passive systems, and low energy consumption and its simplicity compared to Full-Active suspension made it a more popular option that can be adopted into more vehicles.

2.1.7. Problem Statement for Active Suspension

Although active suspension systems are widely used in modern automotive engineering, their practical implementation is rarely emphasized in undergraduate education. Advanced control techniques such as the Linear Quadratic Regulator (LQR) are often studied through simulations, which assume ideal operating conditions and do not capture real-world effects such as friction, actuator limitations, and sensor noise.

As a result, students often struggle to translate theoretical control designs into reliable behavior when applied to real hardware. This challenge is further compounded by the fact that existing active suspension experimental platforms are typically expensive and limited in availability, which restricts meaningful hands-on experience.

This project addresses these limitations by developing a laboratory-scale active suspension experiment that is accessible, cost-effective, and suitable for remote operations. The proposed platform enables the practical validation of control strategies on real hardware while allowing students to directly observe the impact of physical constraints and non-ideal system dynamic.

2.2. Mechanical System

2.2.1. Overall Mechanical Architecture

The active suspension system is implemented as a bench-scale mechanical representation of a quarter-car model, designed to capture the essential vertical dynamics of a vehicle suspension while maintaining experimental flexibility. Our Mechanical Architecture is inspired by widely used educational active suspension platforms, particularly the **Quanser active suspension experiment**, which serves as a reference benchmark in control education.

The developed system adopts a vertically stacked, three-mass configuration, where each mass is constrained to move along a single vertical axis. This configuration allows the suspension dynamics to be studied in a simplified yet physically meaningful form, while remaining suitable for laboratory-scale implementation and remote experimentation.

The mechanical structure consists of three rigid plates arranged vertically:

1. Top acrylic plate (sprung mass): represents the vehicle body
2. Middle acrylic plate (unsprung mass): represents the wheel-tire assembly
3. Bottom plate (road input): represents the road surface excitation

Each plate is guided by parallel vertical shafts and supported by linear bearing blocks, ensuring that motion is strictly translated along the vertical axis with negligible lateral displacement

This arrangement allows the system to physically emulate the classical quarter-car suspension model, where:

- Relative displacement between the top and middle plates corresponds to suspension travel
- Relative displacement between the middle and bottom corresponds to tire deflection

2.2.2. Actuation system

The actuation system is responsible for actively injecting energy into the suspension mechanism to regulate the vertical motion of the masses and counteract road-induced disturbances. In contrast to passive suspension elements, the actuator enables direct control over the system dynamics.

2.2.2.1. Actuator Type and Placement

A brushed DC motor of type **775** is employed as the active actuator in the suspension system. This motor type was selected due to its high rotational speed, compact form factor, availability, and suitability for laboratory-scale experiments. In an active suspension system, the actuator must react within milliseconds to road bumps. The 775 motor provides the necessary "bandwidth" (speed) to move the plates quickly.

Figure 3. 775 DC motor

Motor Type	Operating Voltage	Nominal Voltage	No-Load Speed	Rated Current	Stall Torque	Weight
775 DC Motor	6–20 VDC	12 VDC	12,000 RPM at 12 V	1.2 A at 12 V	79 N.cm at 14.4 V	350 g

Table 1. 775 DC Motor Specifications

2.2.2.2. Suspension Motor Placement

The actuator is mounted on a rigid bracket that is firmly attached to the top plate (sprung mass). The actuator shaft is directly connected to a pulley, which transfers the motor motion through a belt transmission system with a reduction ratio of 2:1. The second pulley is mounted on a horizontal shaft supported by bearings. Through this configuration, the motor primarily influences the relative motion between the vehicle body (sprung mass) and the wheel assembly (unsprung mass), enabling active regulation of suspension travel and body motion.

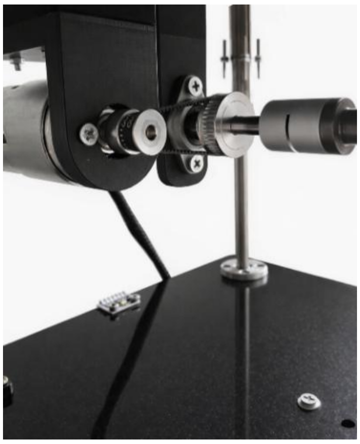


Figure 4. Suspension Motor Fixation

2.2.2.3. Road-Excitation Motor Placement

The road excitation motor is mounted on a rigid bracket that is firmly fixed to the base of the system, which serves as the stationary reference frame for the experiment. The motor shaft is connected to a pulley that drives a belt transmission system with a reduction ratio of 2:1. The driven pulley is mounted directly on the lead screw, converting the motor's rotational motion into linear vertical displacement of the bottom plate. Through this configuration, the motor generates controlled vertical motion of the road input, allowing different excitation profiles to be applied to the suspension system. This arrangement ensures that road disturbances are introduced independently of the suspension actuation, providing a clear and repeatable means of exciting the system for experimental analysis.

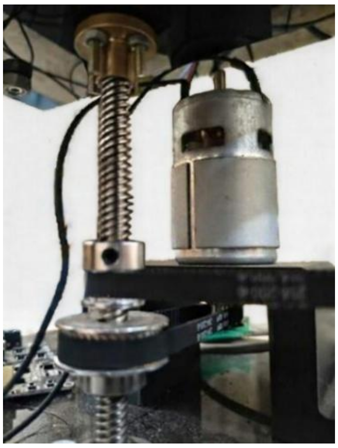


Figure 5. Road Excitation Motor Fixation

2.2.3. Mechanical Transmission

2.2.3.1. Lead Screw Mechanism for Road Excitation

A **lead screw transmission** is employed to generate the vertical displacement that represents road excitation. In this mechanism, motor rotation is converted into precise linear motion through a Leadscrew and nut assembly with pitch 8mm per revolution, resulting in a well-defined and repeatable vertical displacement of the base plate.

The lead screw was selected for this function due to its **high positional accuracy, mechanical stiffness, and repeatability**, which are essential for generating deterministic disturbance profiles such as steps, ramps, or sinusoidal inputs. Ensuring that the imposed input is known and independent of the suspension response.

Figure 6. Lead Screw Mechanism

2.2.3.2. Capstan Mechanism for Active Actuation

The active suspension actuator utilizes a capstan (cable–drum) transmission to convert the rotary motion of the DC motor into a bidirectional linear force applied along the suspension axis. In this configuration, motor rotation drives a cylindrical drum with radius ($r = 6$mm) around which a tensioned cable is wrapped and routed through guiding elements to the moving suspension plate. Rotation of the drum results in a controlled change in cable length, producing upward or downward linear actuation depending on the direction of rotation.

This mechanism was selected due to its ability to deliver smooth and continuous force transmission with minimal backlash when properly tensioned. The compliant nature of the cable transmission also allows the actuator to interact naturally with the suspension dynamics, making it well suited for applying the active control force between the sprung and unsprung masses.

Figure 8. Capstan Mechanism

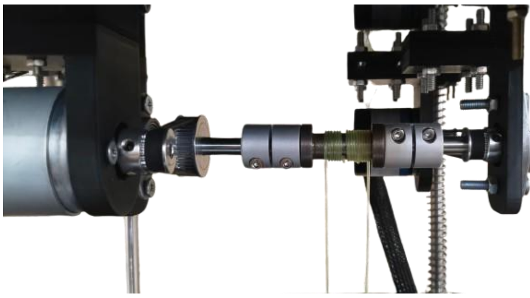


Figure 7. Cable-Drum

2.2.4. Sensors Selection

The actuation system sensing is essential for the experimental investigation of active suspension dynamics. The sensors used in this experiment provide direct measurements of key physical quantities such as displacement, acceleration, and actuator motion, which are necessary for characterizing system response, validating dynamic models, and enabling closed-loop control implementation.

2.2.4.1. Displacement Measurement

Vertical displacement within the suspension system is measured using two Time-of-Flight (ToF) distance sensors, selected for their non-contact operation and ease of integration.

One ToF sensor is mounted on the **bottom plate** facing up to measure the relative displacement between the bottom plate and the middle plate, while the second sensor is mounted on the **middle plate** facing up to measure the relative displacement between the middle plate and the top plate

Enabling evaluation of suspension deflection without relying on mechanical linkages or contact-based sensors. The use of non-contact ToF sensing avoids additional friction or mass loading on the moving components, preserving the natural dynamics of the system.

2.2.4.1.1. Sensor Selection & Specifications

The selected ToF sensor is based on the **VL53L0X** ranging module developed by **STMicroelectronics**, which is widely used in precision short-range distance measurement applications.

Parameter	Specification
Measurement range	Up to ~2 m
Resolution	Up to 1 mm
Typical accuracy	±2 mm
Sampling frequency	Up to 50 Hz
Communication interface	I²C
Operating voltage	2.8–5 V (via onboard regulation)
Field of view	Narrow (focused optical cone)

Table 2. VL53L0X Specifications

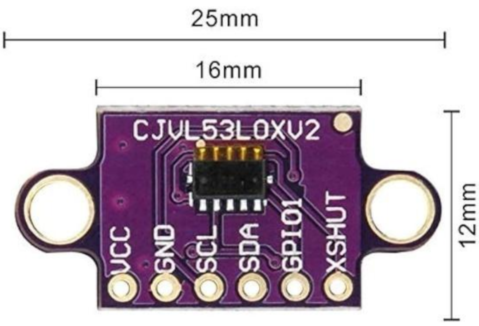


Figure 9. TOF Sensor

The operating principle of a ToF sensor is based on the emission of a modulated infrared light pulse toward a target surface and the subsequent detection of the reflected signal. The sensor measures the time delay between emission and reception, which is proportional to the distance traveled by the light. By internally processing this time delay, the sensor computes the distance to the target with high repeatability over the operating range relevant to the suspension travel.

2.2.4.1.2. Sensor Placement & Measurement Configuration

Two ToF sensors are used in the experimental setup:

- Sensor 1: Mounted on the **bottom plate**, measuring the vertical displacement of the middle plate (road excitation input)
- Sensor 2: Mounted on the **middle plate**, measuring the vertical displacement of the top plate (sprung mass motion)

Each sensor is aligned along the vertical axis, with its optical path perpendicular to the target surface.

2.2.4.2. Acceleration Measurement

Accurate measurement of plate acceleration is essential in the active suspension experiment, as acceleration is directly related to ride comfort and is a primary performance metric in suspension control systems. Unlike displacement measurements, which reflect relative motion, acceleration captures the dynamic response of the system to road disturbances and control actions in real time.

For this reason, an **inertial measurement unit (IMU)** was employed to measure the vertical acceleration of both the middle plate (unsprung mass) and the top plate (sprung mass). These measurements provide direct insight into vibration transmission through the suspension stages and enable quantitative evaluation of controller performance.

2.2.4.2.1. *Sensor Selection & Specifications*

The **MPU6050** was selected as the acceleration sensor for this experiment due to its suitability for laboratory-scale dynamic systems and its balance between performance, cost, and ease of integration. The sensor integrates a 3-axis MEMS accelerometer and a 3-axis gyroscope within a single compact package, allowing synchronized inertial measurements with minimal wiring and computational overhead.

Parameter	Specification
Accelerometer axes	3 (X, Y, Z)
Measurement range	±2 g, ±4 g, ±8 g, ±16 g (configurable)
Accelerometer sensitivity	16,384 LSB/g (±2 g mode)
Sampling frequency	Up to 1 kHz (internal)
Communication interface	I²C
Operating voltage	3.3 V (5 V tolerant via onboard regulator)
On-chip features	Digital Low-Pass Filter (DLPF), FIFO buffer

Table 3. MPU6050 Specifications

Figure 10. MPU6050 Sensor

For this experiment, the accelerometer was configured to operate in the ±2 g range, maximizing resolution for small-amplitude vertical vibrations typically observed in suspension systems.

2.2.4.2.2. *Sensor Placement & Measurement Configuration*

Two MPU6050 sensors are used in the system:

- Sensor 1: Mounted on the middle plate to measure the acceleration of the unsprung mass
- Sensor 2: Mounted on the top plate to measure the acceleration of the sprung mass

Both sensors are rigidly attached to their respective plates to ensure accurate transmission of vertical motion. The sensor axes are aligned such that the Z-axis corresponds to the vertical direction of motion, while the remaining axes are ignored in software.

2.2.4.3. Actuator Motion Measurement

Encoder feedback is used in the experimental setup to provide precise information about the rotational motion of the actuators driving the suspension and road excitation mechanisms.

2.2.4.3.1. *Sensor Selection & Specifications*

Each motor is equipped with a **high-resolution incremental encoder with 2400 pulses per revolution (PPR)**, providing fine angular resolution over the full rotational range.

The encoder provides two quadrature pulse signals that are processed by the control system to extract the following quantities:

- Motor Position: Determined by counting encoder pulses relative to a reference position, providing angular displacement of the motor shaft.

This quantity is directly related to the linear motion of the suspension plates through the mechanical transmission mechanisms (capstan or lead screw), allowing indirect observation of system actuation behavior.

2.2.5. Stroke limitation and mechanical Safety

To protect the system from over-travel and mechanical damage, we ensured:

- Defined mechanical stroke limits.
- End-of-travel detection via limit switches
- Structural clearance to prevent spring over-compression

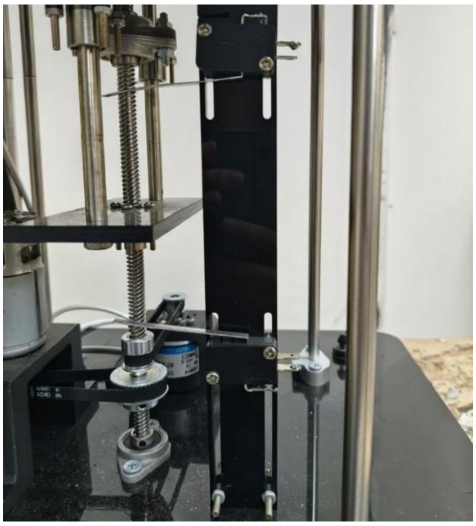


Figure 11. Limit switches for mechanical safety

2.2.6. CAD Modeling and Assembly

The CAD models were developed using SolidWorks and serve as an accurate geometric representation of the finalized mechanical design. The objective of the CAD work is to document the physical realization of the system, verify component integration, and support assembly, alignment, and future modifications.

All mechanical components were modeled individually and subsequently assembled to replicate the real mechanical constraints and degrees of freedom present in the physical system.

2.2.6.1. Full Assembly

The complete CAD assembly of the active suspension system is shown in **Figure 12.** The model illustrates the stacked plate configuration, guiding columns, and spring–damper elements that define the system's vertical motion. This view highlights the overall mechanical architecture and the integration of the actuation and excitation subsystems within a rigid and modular frame.

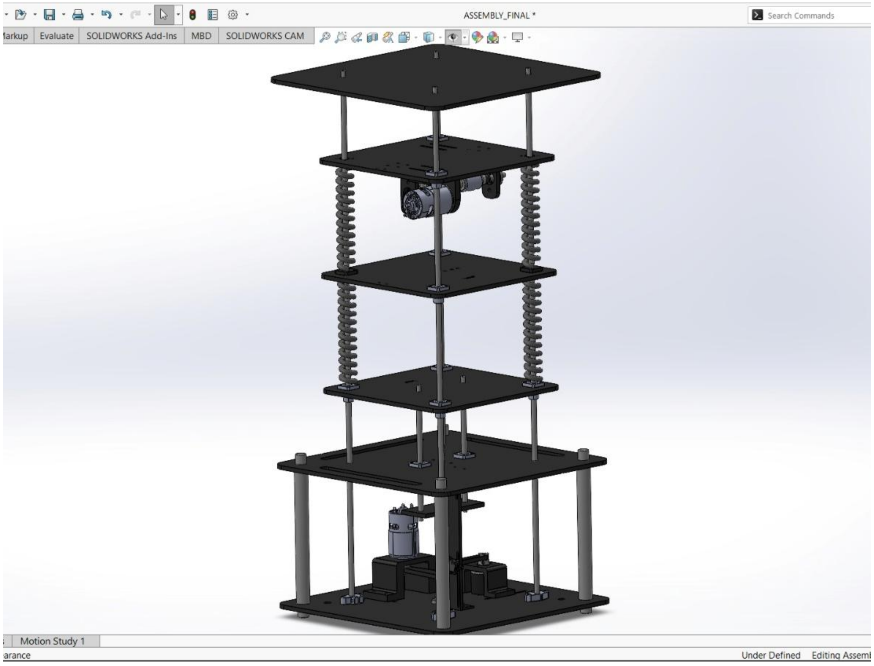


Figure 12. CAD Assembly

2.2.6.2. Capstan mechanism

Figure 13 presents the capstan-based actuation mechanism used for active suspension control. The motor is mounted on a rigid bracket, and its rotational motion is transmitted through a belt-driven capstan arrangement to generate controlled linear force between the upper and middle plates. This configuration ensures smooth force transmission and reduces backlash while maintaining proper shaft alignment.

Figure 13. Capstan Mechanism Assembly

2.2.6.3. Road Excitation Mechanism

The road excitation mechanism is illustrated in **Figure 14**. A DC motor mounted on the base drives a vertical lead screw through a belt–pulley transmission, producing controlled vertical displacement of the bottom plate. This arrangement allows accurate generation of road disturbance profiles while isolating the motor from axial loading.

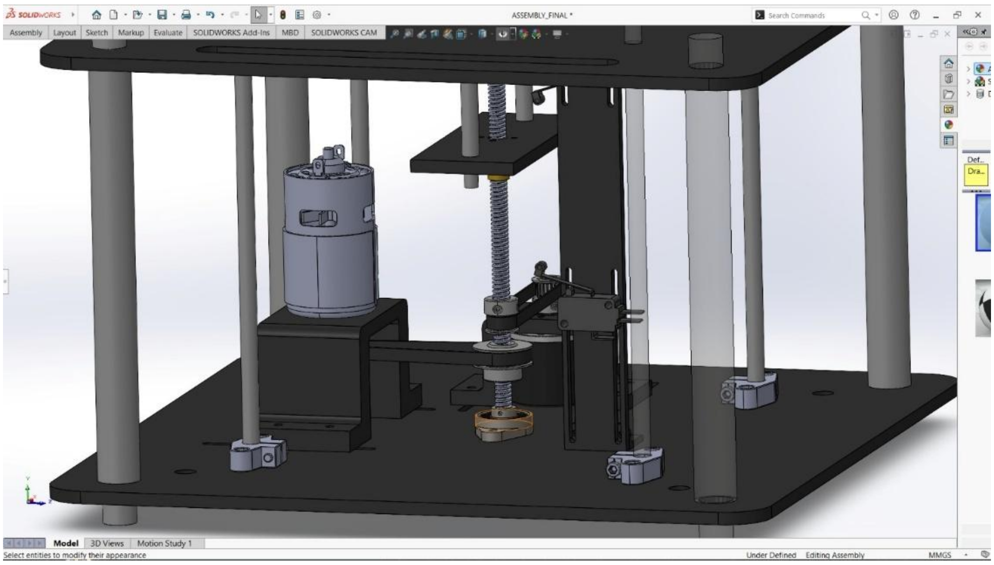


Figure 14. Road Excitation Mechanism Assembly

.

2.3. Electrical system

2.3.1. Electrical System Overview

The electrical system provides the interface between the mechanical plant and the control environment. It is responsible for powering the actuators, interfacing all sensors, and enabling real-time communication between the physical system and the controller. The design emphasizes robustness, compatibility with real-time simulation, and seamless integration with the Simulink-based control framework used throughout the experiment.

All sensing, actuation, and signal processing tasks are implemented exclusively within the **Simulink environment**, without the use of external microcontroller IDEs. This approach ensures consistency between simulation and experimental implementation and supports the educational objectives of the laboratory.

2.3.2. Power Supply and Distribution

The system is powered using a controllable DC power supply with an adjustable output range of **12–24 V,** which serves as the main energy source for the experiment. This supply directly feeds the motor drive electronics to meet the high current demands of the DC motors during dynamic operation.

Figure 15. Power Supply

Power distribution is separated into high-power and low-power domains. The high-power path supplies the **Cytron** motor drivers, which in turn drive the DC motors. Low-power electronics, including the control hardware and sensors, are supplied through a dedicated **DC–DC converter** that steps down the main supply voltage to a regulated **3.3 V** rail suitable for the ESP32 and logic-level devices.

A common ground reference is maintained across all system components to ensure reliable signal integrity, particularly for digital communication interfaces and encoder signals. This separation between power electronics and low-power logic improves system stability and reduces electrical noise coupling.

2.3.3. Actuator Drive Electronics

Actuation is achieved using DC motors driven by Cytron motor drivers, which are external to the control hardware. The Cytron drivers were selected due to their ability to withstand high current levels and their minimal voltage drop, ensuring efficient power delivery to the motors. Electrically, motor drivers function as power amplification stages. They accept low-power control signals generated by the controller, such as PWM and direction signals, and convert them into high-current motor drive signals. This configuration isolates the control hardware from the electrical stresses associated with motor operation and ensures safe current handling under varying load conditions.

Figure 16. Cytron MDD10A Motor Driver

2.3.4. Wiring and Interconnection Architecture

This section describes the electrical wiring and interconnection architecture of the system, focusing on how power and signal lines are distributed between actuators, sensors, motor drivers, and the control hardware. The objective is to clarify the electrical connectivity at a system level without delving into low-level schematic details.

2.3.4.1. Motor and Driver Connections

Two DC motors are driven through the Cytron motor driver, with each motor connected to a dedicated driver channel. The Cytron driver is powered directly from the controllable 12–24 V DC power supply, enabling it to deliver the required current to both motors under dynamic operating conditions.

Control signals generated by the ESP32 are routed to the Cytron driver inputs, while the high-current motor connections remain isolated from the low-power control circuitry.

2.3.4.2. Encoder Power and Signal Wiring

Two incremental encoders are used for motion measurement. Encoder power (VCC and GND) is supplied directly from the main power supply to meet the encoder operating requirements. The quadrature output signals (Phase A and Phase B) are interfaced with the ESP32 through external **1.2 kΩ pull-up resistors**, which pull the signal lines up to **3.3 V**.

This configuration ensures compatibility with the ESP32's logic levels while maintaining reliable signal integrity and accurate quadrature decoding within the Simulink environment.

2.3.4.3. Sensor Interconnection via ESP32 Shield

The inertial sensors (MPU) and time-of-flight (ToF) displacement sensors are mounted and interfaced through an **ESP32 PCB shield**, which provides organized routing for power and communication lines. All sensors are powered from the regulated **3.3 V** rail supplied by the ESP32.

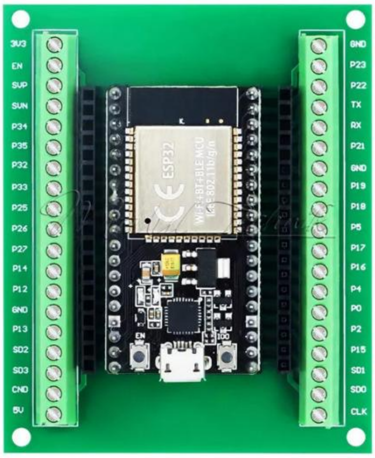


Figure 17. ESP32 shield

Communication between the sensors and the controller is achieved via the I²C bus, with the SDA and SCL lines connected directly to the corresponding ESP32 I²C pins through the shield. This shared bus architecture reduces wiring complexity and supports synchronized data acquisition for all sensors.

2.3.4.4. Control Hardware Power Path

The ESP32 control hardware receives its operating voltage from a DC–DC converter, which steps down the main power supply voltage to a regulated 3.3 V output. This converter is fed directly from the controllable DC power supply, ensuring stable operation of the control hardware and all connected sensors.

A common ground reference is maintained across the power supply, motor drivers, control hardware, and sensors to ensure reliable signal communication and consistent system behavior.

Figure 18. DC-DC Step-Down Converter

2.3.4.5. Wiring Architecture Overview

Figure 19 illustrates the overall wiring and interconnection architecture of the electrical system, showing the distribution of power and signal paths between the power supply, motor drivers, control hardware, actuators, and sensors. The diagram emphasizes the separation between high-current actuation circuits and low-power logic and sensing circuits, while maintaining a unified control structure centered around ESP32.

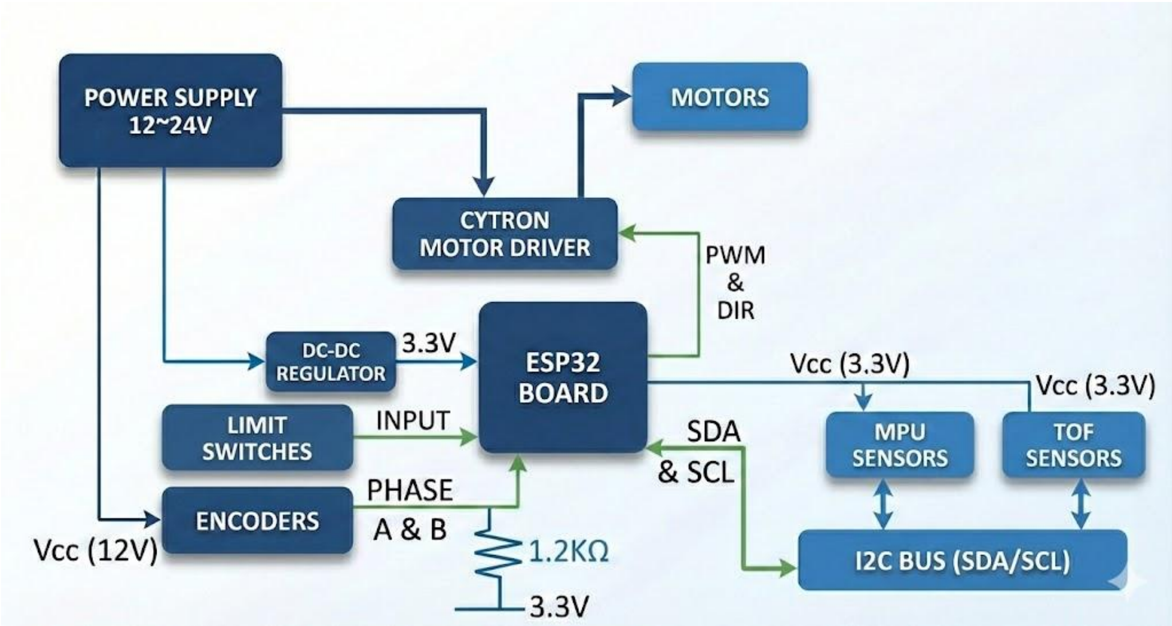


Figure 19. Wiring Architecture

2.3.4.6. Sensor Interface and Signal Conditioning

All displacement and inertial sensors used in the system are digitally interfaced with the controller through a shared I²C communication bus. This includes the Time-of-Flight (ToF) distance sensors and the inertial measurement units (MPUs), both operating at 3.3 V logic levels and connected to the ESP32 via a dedicated PCB shield that provides power distribution and bus route.

2.3.4.6.1. Multi-Sensor I²C Addressing and Initialization Strategy

Since multiple I²C devices coexist on a single communication bus, special consideration was given to sensor initialization and address management. While the inertial (MPU) sensors support hardware-selectable I²C addresses, the Time-of-Flight (ToF) displacement sensors do not provide a dedicated hardware pin for static address selection. As a result, the simultaneous use of two identical ToF sensors introduces an addressing challenge.

To enable reliable multi-sensor operation, the ToF sensors require a shutdown-based (XSHUT) startup sequence that allows each device to be enabled sequentially and assigned a unique I²C address during system initialization. Implementing this timing-critical and state-dependent sequence using standard Simulink blocks is not ideal.

To overcome this limitation, a custom **Simulink S-Function** was developed and integrated into the model. This S-Function encapsulates the complete XSHUT control and address reassignment procedure, ensuring deterministic sensor bring-up at startup. Once initialization is completed, normal sensor data acquisition proceeds through standard Simulink blocks.

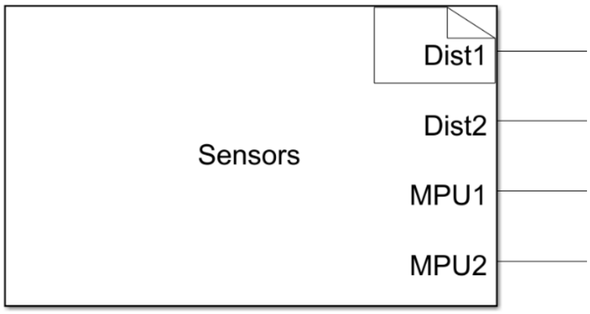


Figure 20. Sensors S-Function Block

2.3.4.6.2. Signal Filtering

From an electrical perspective, all sensor signals are digital and therefore do not require analog signal conditioning hardware. Instead, signal conditioning is performed entirely within Simulink, where low-pass filtering blocks are applied uniformly to both ToF and IMU measurements to attenuate high-frequency noise prior to control processing. This approach maintains a clean and modular electrical design while allowing signal conditioning parameters to be adjusted directly within the control environment.

2.4. Mathematical model & System Identification

2.4.1. Dynamic Model

We want to design a control system for an active suspension. We limited our study to a simple 2-DOF quarter car model. We will develop a mathematical model of the quarter car problem that we can use as a basis to understand the vertical movement of the vehicle.

2.4.2. Model Parameters

The Model consists of:

- Sprung mass: M_s $[Kg]$
- Unsprung mass: M_{us} $[Kg]$
- A suspension spring–damper pair (k_s, B_s) connecting the sprung mass to the unsprung mass.
- A tire spring–damper pair (k_{us}, B_{us}) connecting the unsprung mass to the road surface.
- The active suspension control command: F_c applied between the sprung and unsprung masses.
- A road displacement input $z_r(t)$.

i.e. The generalized coordinates x1 and x2 respectively.

- represents the tire displacement (usnprung mass in quarter car model).
- represents the vehicle body displacement (sprung mass in the quarter car model).

all with respect to the ground. The positive directions are upwards.

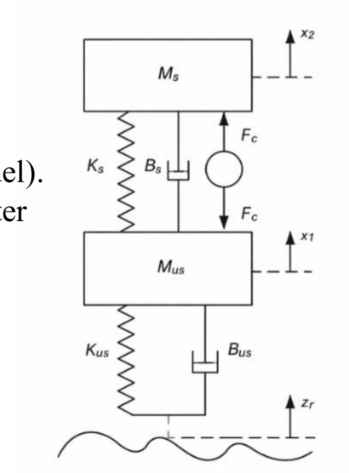


Figure 21. Double Mass-Spring-Damper used to model Active Suspension

Assumption:
All displacements are defined with respect to the static equilibrium position of the system. Consequently, gravitational forces are balanced by spring preload and do not appear in dynamic equations.

2.4.3. Equation of Motion

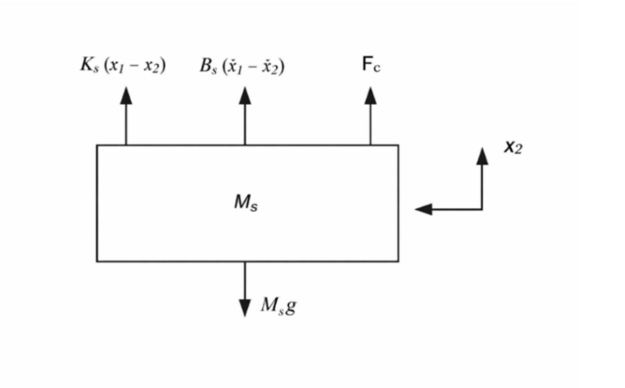


Figure 22. Th Free Body Diagram of M_s

The EOM for M_s will be as follows:

$$\ddot{x}_2 = -g + \frac{F_c}{M_s} + \frac{B_s\dot{x}_1}{M_s} - \frac{B_s\dot{x}_2}{M_s} + \frac{K_s x_1}{M_s} - \frac{K_s x_2}{M_s} \qquad (2.1)$$

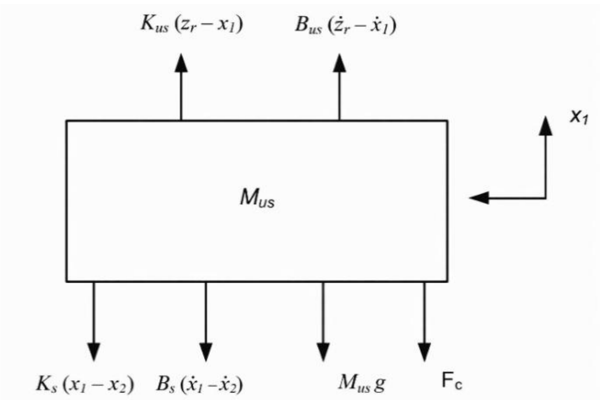


Figure 23. Th Free Body Diagram of M_{us}

The EOM for M_{us} will be as follows:

$$\ddot{x}_1 = -g - \frac{F_c}{M_{us}} - \frac{(B_s + B_{us})\dot{x}_1}{M_{us}} + \frac{B_s\dot{x}_2}{M_{us}} + \frac{B_{us}\dot{z}_r}{M_{us}} - \frac{(K_{us} + K_s)\,x_1}{M_{us}} + \frac{K_s x_2}{M_{us}} + \frac{K_{us} z_r}{M_{us}} \qquad (2.2)$$

2.4.3.1. Eliminating Gravity force from EOM

The objective of this section is to prove mathematically that the gravity force only changes the equilibrium points in the Active Suspension EOM, and it does not affect the dynamics of the system.

i.e. $x_1 = x_{eq1}, \ x_2 = x_{eq2}$

Substituting these changes in equations 2.1 and 2.2 will give the following results

$$M_s g + K_s(x_{eq2} - x_{eq1}) = 0 \tag{2.3}$$

$$M_{us} g + K_s(x_{eq1} - x_{eq2}) + K_{us} x_{eq1} = 0 \tag{2.4}$$

As a result, the equilibrium points due to gravity will be:

$$x_{eq1} = -\frac{g(\,M_s + M_{us})}{K_{us}} \tag{2.4}$$

$$x_{eq2} = -\frac{g(\,M_s K_{us} + K_s M_{us} + K_s\,M_s)}{K_{us} K_s} \tag{2.5}$$

To remove gravity forces from the equations of motion we apply the following change of variables to the equations of motion:

$$x_1 = z_{us} - \frac{g(\,M_s + M_{us})}{K_{us}} \tag{2.6}$$

$$x_2 = z_s - \frac{g(\,M_s K_{us} + K_s M_{us} + K_s\,M_s)}{K_{us} K_s} \tag{2.7}$$

$$\dot{x}_1 = \dot{z}_{us}\,, \dot{x}_2 = \dot{z}_s \tag{2.8}$$

$$\ddot{x}_1 = \ddot{z}_{us}, \ddot{x}_2 = \ddot{z}_s \tag{2.9}$$

i.e. $z_{us} = 0$, $z_s = 0$, will be the equilibrium point of the system.

From Newton's Second Law:

$$\Sigma F_z = m\ddot{z} \tag{2.10}$$

For Sprung Mass:

$$M_s \ddot{z}_s + c_s\,(\dot{z}_s - \dot{z}_{us}) + K_s(z_s - z_{us}) = F_c \tag{2.11}$$

For Unsprung Mass:

$$M_{us}\ddot{z}_{us} + B_s(\dot{z}_{us} - \dot{z}_s) + B_{us}(\dot{z}_{us} - \dot{z}_r) + K_s(z_{us} - z_s) + K_{us}(z_{us} - z_r) = -F_c \qquad (2.12)$$

2.4.3.2. Mapping of Actuator Force

Although the active suspension actuator applies directly to the system, the experimental platform does not provide direct force actuation or measurement. Instead, the system is excited through velocity commands, which are accurately generated and measured.

Starting from (2.12) Expanding the tire damping term yields:

$$-B_{us}(\dot{z}_{us} - \dot{z}_r) = -B_{us}\dot{z}_{us} + B_{us}\dot{z}_r \qquad (2.13)$$

Substituting (2.12) into (2.13) gives:

$$M_{us}\ddot{z}_{us} = -B_s(\dot{z}_{us} - \dot{z}_s) - K_s(z_{us} - z_s) - K_{us}(z_{us} - z_r) - B_{us}\dot{z}_{us} + B_{us}\dot{z}_r - F_c \qquad (2.14)$$

The system is being excited using the road velocity $\dot{z}_r$, which generates an equivalent force through the tire damping term. By setting $F_c = 0$, the equivalent force reduces to:

$$F_{eq} = B_{us}\dot{z}_r \qquad (2.15)$$

2.4.4. State space Model:

2.4.4.1. Definition:

Representing a set of linear differential equations that describe the system's dynamics. The two EOM can be written under the state-space representation as follows:

2.4.4.2. State space Representation

The state space approach is a convenient way to model the quarter-car model with multiple inputs and outputs. The states can be defined as such that they reflect the system performance parameters that have to be optimized as this state space can be later used for state-feedback controller and observer design.

Due to the existence of four energy storage elements in the quarter model; four states for the system.

State Vector:

$$X = \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix} = \begin{bmatrix} z_s - z_{us} \\ \dot{z}_s \\ z_{us} - z_r \\ \dot{z}_{us} \end{bmatrix} \qquad (2.16)$$

Where,

- x_1 : suspension deflection/travel. (the gap between body and wheel)
- x_2 : vehicle body vertical velocity.
- x_3 : tire deflection which is a measure of road handling. (the gap between wheel and road)
- x_4 : tire vertical velocity.

Input Vector:

$$u = \begin{bmatrix} u_1 \\ u_2 \end{bmatrix} = \begin{bmatrix} Fc \\ \dot{z}_r \end{bmatrix} \tag{2.17}$$

Where,

- u_1 : Suspension Force
- u_2 : Road excitation velocity.

Output Vector:

$$y = \begin{bmatrix} z_s - z_{us} \\ z_{us} - z_r \end{bmatrix} \tag{2.18}$$

Where,

- The first measured output is the suspension travel.
- The Second measured output is the Tire deflection.

Using the EOMs, we can calculate matrices A, B, C, and D as follows:

A =

$$\begin{bmatrix} 0 & 1 & 0 & -1 \\ -\frac{K_s}{M_s} & -\frac{B_s}{M_s} & 0 & \frac{B_s}{M_s} \\ 0 & 0 & 0 & 1 \\ \frac{K_s}{M_{us}} & \frac{B_s}{M_{us}} & -\frac{K_{us}}{M_{us}} & -\frac{(B_s + B_{us})}{M_{us}} \end{bmatrix} \tag{2.19}$$

$$B = \begin{bmatrix} 0 & 0 \\ 0 & 0 \\ 0 & -1 \\ 0 & \frac{B_{us}}{M_{us}} \end{bmatrix} \tag{2.20}$$

$$C = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix}, D = \begin{bmatrix} 0 & 0 \\ 0 & 0 \end{bmatrix} \tag{2.21}$$

2.5. Linear Quadratic Regulator (LQR)

2.5.1. Introduction

We know from the previous chapter that Full State Controllability implies that we can arbitrarily place the closed loop poles of the system using suitable feedback matrix equation (4.1).

The intuition may say to place them far into the left half plane. This may sound intuitive, but if we discuss this idea further. We will realize that driving the poles further into Left half plane will result in significantly larger gain Matrix K that will result in unreasonable control signal that can't be met by the actuator.

Then we need a form of experience to pick a pole location that balances fast response of the system and the limits on actuator signal.

2.5.2. LQR Problem Formulation

The LQR formulation defines a cost function J that penalizes both state deviation and control signal. (Q and R) are diagonal weighting matrices

$$\int_0^{\infty} X^T\, Q\, x + u^T\, R\, u \;\; dt \tag{2.22}$$

Then we can solve an optimization problem to decide the feedback gains that result in a minimum cost.

The true power of LQR lies in simplicity in tuning because Tuning parameters (The Q and R Matrices) have a meaningful physical interpretation which makes the tuning process far easier and more intuitive.

2.5.3. Derivation of Optimal Gain

The solution to the optimization problem is that

$$u = -Kx$$

$$\text{where } K = R{-}1BT\ S$$

where S is the solution to the Algebraic Riccati Equation

$$AT\ S + SA - SBR{-}1BT\ S + Q = 0$$

Detailed proof for solving the optimization problem and the Riccati Equation can be obtained from many textbooks.

2.5.4. LQR Design Process

Now the design process is more systematic. In original pole placement formulation, the process start as follows:

- Decide on Closed Loop pole location
- Obtain Feedback Matrix using Ackermann Formula or comparing the coefficients for low dimensional Systems.

Using the LQR formulation. The process changes to

- Select relative importance of each state and every actuator
- Solve the algebraic Riccati Equation for Feedback matrix

2.5.5. LQR and State Estimation

Both pole placement techniques and LQR require the full state vector x to be measured and fed back. This case is rarely realized in actual applications either because it's not economic to use that many sensors to measure all these states or some state cannot be measured at all.

So, in the actual application we end up with partial measurements of the state vector, and we are required to find a way to reconstruct the full state vector from that measurement.

If the system is Observable, then we can build a state observer (State Estimator) that takes in the incomplete state vector and reconstructs the full state vector to be used in feedback. One such example of state observers is the Kalman Filter which will be discussed in further detail in the following chapter.

The separation of control and observation principle allows to design an optimal state observer (minimizes error) and optimal control (minimizes a cost function) and the combined system of the controller, and the state observer is still optimal which makes the development straightforward. One important combination is the Kalman filter and LQR is called the Linear Quadratic Gaussian (LQG)

2.6. Simulation and Results

2.6.1. Introduction

This chapter presents the simulation framework and performance evaluation of the active suspension system. The objective of the simulation is to validate the dynamic model of the system and assess the effectiveness of the proposed control strategy under representative road excitation conditions.

The simulation environment is developed using a combination of Simscape and Simulink, where Simscape is employed to model the physical dynamics of the suspension system, while Simulink is used for control implementation, signal processing, and result analysis. This integrated approach ensures consistency between the physical system description and the control-oriented simulation.

The chapter is organized as follows: first, the simulation framework and model architecture are presented, including the plant and controller subsystems. This is followed by the definition of the reference input and control parameters. Finally, simulation results are presented and discussed in terms of key performance metrics such as control effort, suspension travel, road handling, and ride comfort.

2.6.2. Simulation Framework Overview

The simulation framework of the active suspension system is developed using an integrated **SolidWorks–Simulink–Simscape** workflow to ensure strong consistency between the mechanical design and the dynamic model. The CAD model of the system is first created in SolidWorks and then imported into Simulink, where it is used as the basis for constructing the Simscape representation of the mechanical plant.

Simscape is employed to model the physical dynamics of the system using energy-based components that represent masses, springs, and force interactions. Simulink is used to implement the control logic, reference inputs, signal conditioning, and performance evaluation. This integrated approach allows the mechanical structure, physical dynamics, and control system to be developed and evaluated within a unified simulation environment that closely reflects the real experimental setup.

2.6.3. Simscape-Based Plant Modeling

The Simscape–Simulink model represents the physical realization of the active suspension system and forms the core of the simulation framework. The mechanical plant is implemented using Simscape components derived from the imported SolidWorks CAD model, ensuring that the simulated dynamics are consistent with the physical geometry and constraints of the system. Simulink is used alongside Simscape to manage signal routing, sensing interfaces, force actuation inputs, and data extraction, resulting in a unified physical–control modeling environment.

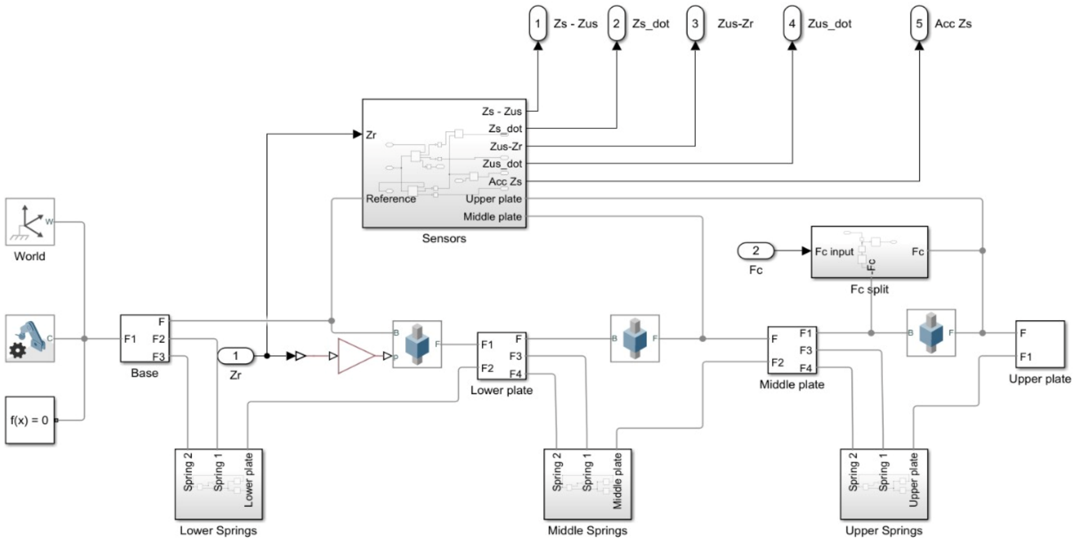


Figure 24.Simulnk-Simscape

Figure 24 shows the Simscape–Simulink representation of the active suspension system. The model consists of the base, lower plate, middle plate, and upper plate, interconnected through spring elements that represent the suspension stages. Road excitation is applied at the base through the displacement input Z_r, while the active control force F_c is injected into the system and distributed using a force-splitting block. Sensor outputs such as suspension deflection, relative velocities, and sprung-mass acceleration are extracted from the plant and routed to output ports for feedback and performance evaluation.

2.6.4. Overall Simulink Model Architecture

The overall Simulink model integrates the physical plant, controller, reference input, and performance evaluation blocks into a complete closed-loop simulation framework. This level of the model emphasizes system-level signal flow and interaction between subsystems rather than internal physical details. The modular structure allows the plant, controller, and reference generation to be developed and tested independently while maintaining a coherent overall architecture.

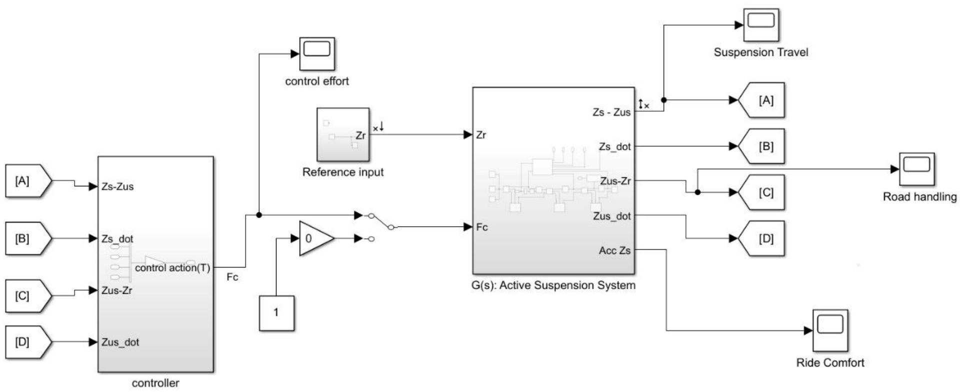


Figure 25. Simulink Model Architecture

Figure 25 illustrates the overall Simulink architecture of the active suspension system. The reference input block generates the road disturbance signal, which is applied to the plant subsystem. Measured outputs from the plant are fed into the controller block, where the control action is computed and returned to the plant as the active force input F_c. Dedicated output blocks are used to monitor key performance metrics, including suspension travel, road handling, ride comfort, and control effort, enabling direct comparison of system behavior under different operating conditions.

2.6.5. Plant Subsystem Description

The plant subsystem represents the complete physical model of the active suspension system and is implemented entirely using Simscape components. It encapsulates the mechanical dynamics derived from the imported CAD model, including the masses, elastic elements, and force interactions that govern system behavior. The subsystem is structured to mirror the physical assembly, ensuring a clear correspondence between simulated components and their real-world counterparts.

Within the plant subsystem, the system is divided into modular elements representing the base, lower plate, middle plate, and upper plate. These elements are interconnected through spring components corresponding to the suspension stages. Road excitation is introduced at the base via the displacement input Z_r, while the active control force $F_c$is applied through a dedicated force input and distributed internally using a force-splitting mechanism. Sensor interfaces embedded within the plant extract relevant physical quantities such as suspension deflection, relative velocities, and acceleration, which are routed to the controller and output ports for monitoring and evaluation.

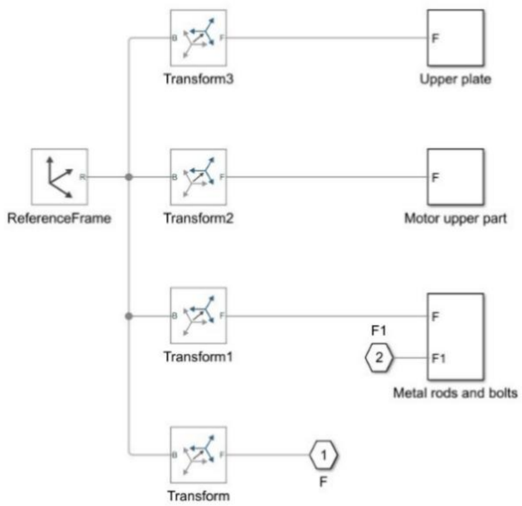


Figure 26. Upper Plate Subsystem

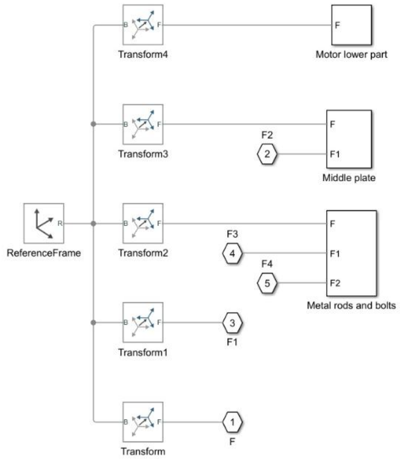


Figure 27. Middle Plate Subsystem

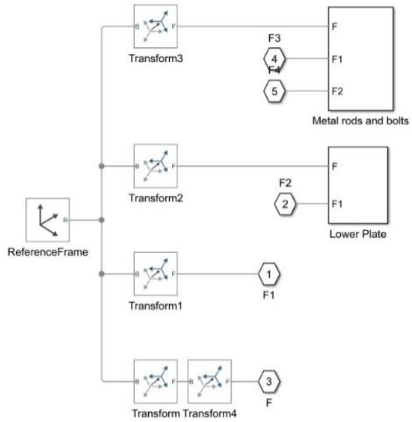


Figure 28. Lower Plate Subsystem

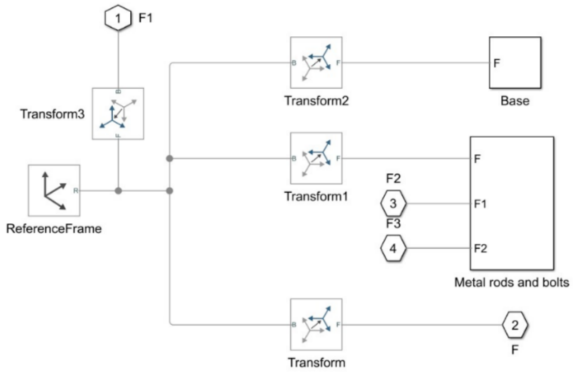


Figure 29. Base Subsystem

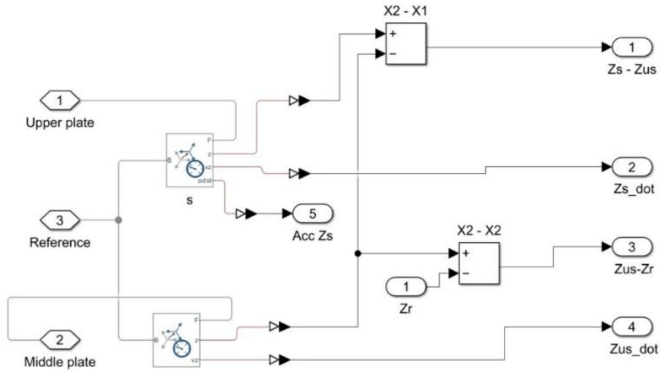


Figure 30. Sensors Subsystem

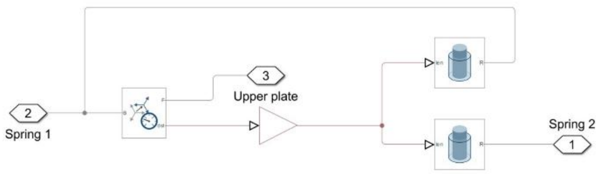


Figure 31. Springs Subsystem

2.6.6. Controller Subsystem Description

The controller subsystem is responsible for generating the active control force applied to the suspension system based on measured system states. It operates within the Simulink environment and interfaces directly with the plant subsystem through defined input and output signals. The controller receives feedback signals representing key dynamic variables of the suspension system and computes the corresponding control action to regulate system behavior.

The control strategy implemented in this work is based on a Linear Quadratic Regulator (LQR) formulation. The controller block processes the measured state variables and outputs a single control force signal F_c, which is applied to the plant as the active actuation input. The controller subsystem is designed as a standalone block, allowing the control law to be modified or replaced without altering the underlying plant model, thereby supporting modularity and extensibility of the simulation framework.

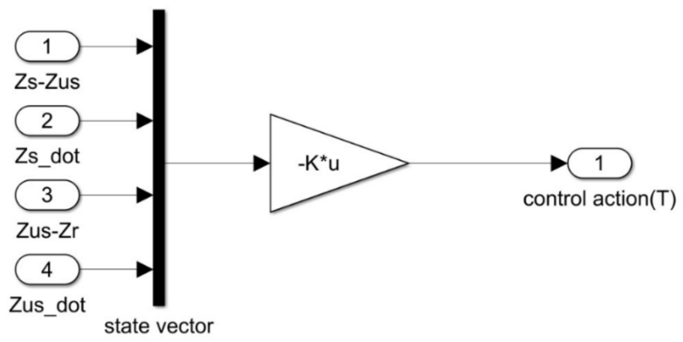


Figure 32. Controller Subsystem

2.6.7. Reference Input and Excitation Profile

The road disturbance applied to the active suspension system is introduced as a vertical displacement input at the base of the model through Z_r. This reference input represents the road excitation used to evaluate the system's dynamic response under controlled conditions.

The excitation signal is defined as:

- **Signal type:** Square wave displacement
- **Amplitude:** 2 cm
- **Frequency:** 1 Hz

This input profile provides a clear and repeatable disturbance, allowing effective assessment of suspension travel, ride comfort, road handling, and control effort.

2.6.8. LQR Design Summary

Item	Value
System Controllability Rank	4/4
LQR Gain Matrix K	[11.3848,61.4922, −206.1923, −4.4806]
Closed-Loop Poles	−14.3835±11.8910i, −20.3057±49.0206i

Table 4. LQR gain and closed loop poles

2.6.9. Simulation Results and Performance Evaluation

The following results compare the system response under passive and active suspension operation. During the **first 5 seconds**, the control action is disabled and the system behaves as a passive suspension. At $t = 5$**seconds**, the LQR controller is activated, and the system operates in active suspension mode. This transition allows direct comparison of system performance before and after control activation under identical excitation conditions.

2.6.9.1. Control Effort

This plot shows the control force generated by the actuator during the active suspension phase. The control effort is zero during the passive interval and becomes nonzero once the controller is enabled, reflecting the force required to mitigate the applied road disturbance.

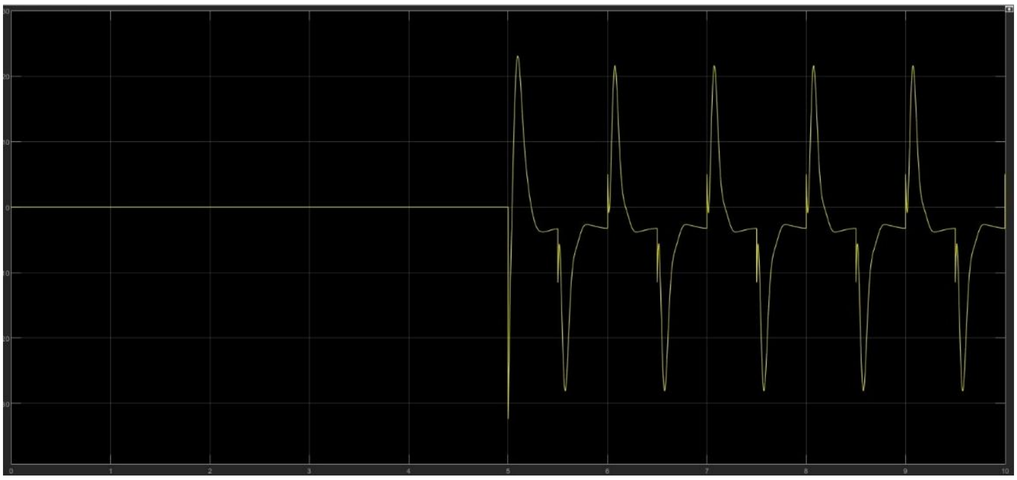


Figure 33. Control Effort

2.6.9.2. Suspension Travel

This plot illustrates the relative displacement between the sprung and unsprung masses. Activation of the controller reduces suspension travel magnitude, indicating improved control of suspension deflection compared to the passive case.

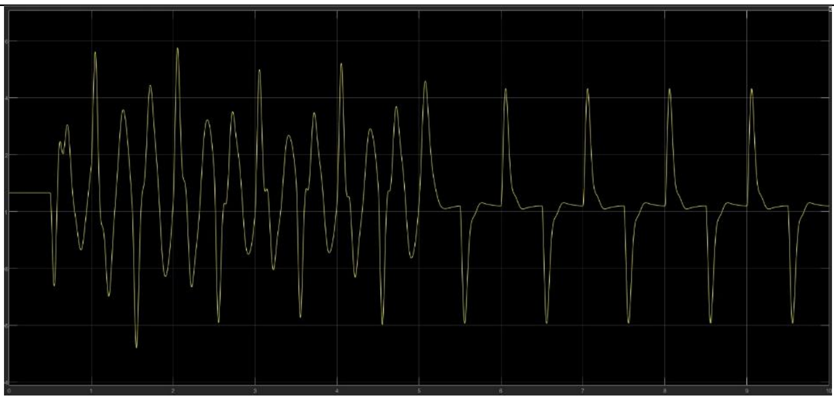


Figure 34. Suspension Travel

2.6.9.3. Road Handling

This plot represents the ability of the suspension system to maintain tire contact with the road surface. Improved road handling performance is observed after controller activation, as indicated by reduced relative displacement between the wheel and road input.

Figure 35. Road Handling

2.6.9.4. Ride Comfort

This plot shows the vertical acceleration of the sprung mass, which is directly related to passenger comfort. The active suspension significantly attenuates acceleration levels compared to the passive response, demonstrating enhanced ride comfort.

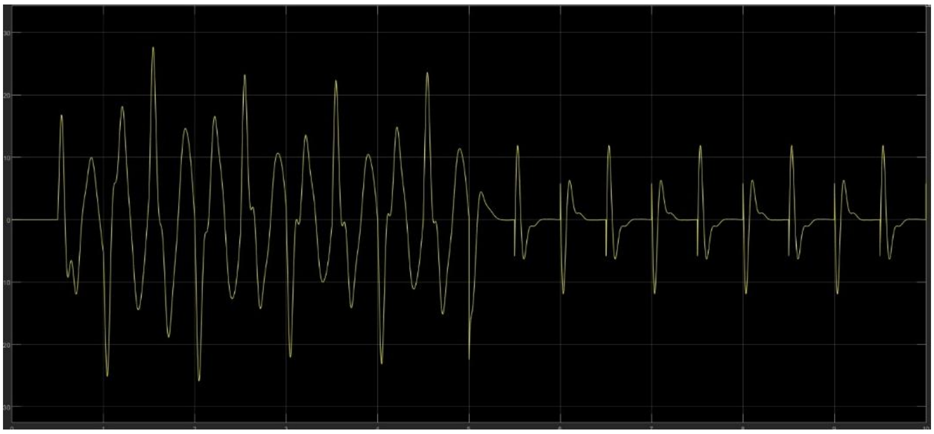


Figure 36. Ride Comfort

2.7. Summary

This chapter presented the design and simulation-based evaluation of the active suspension system. The mechanical and electrical architectures were introduced, followed by the development of a physics-based plant model using a SolidWorks–Simulink–Simscape workflow. An LQR controller was implemented on the simulated plant to actively regulate the suspension response under a representative road excitation. Simulation results demonstrated the effectiveness of active control in improving suspension travel, road handling, and ride comfort compared to passive operation, providing a solid foundation for subsequent experimental implementation.

3. Rotary Inverted Pendulum

3.1. Introduction

3.1.1. Background

The Rotary Inverted Pendulum, commonly referred to as the Furuta Pendulum, is a well-established benchmark system in control engineering due to its nonlinear dynamics, underactuated structure, and unstable equilibrium point. The system exhibits strong coupling between rotational degrees of freedom, making it an ideal platform for studying advanced control strategies beyond classical linear control.

The primary motivation for implementing this system is to experimentally validate modern state-space and optimal control techniques under real-time constraints. The pendulum must be stabilized in its upright position while maintaining robustness against external disturbances and actuator limitations. Achieving this objective requires a structured approach that includes physical modeling, controller synthesis, and experimental verification.

3.1.2. Rotary Inverted Pendulum

The rotary inverted pendulum system consists of a horizontally rotating arm driven by an electric motor and a pendulum link mounted at the end of the arm, free to rotate in the vertical plane. The motor applies torque to the rotary arm, which indirectly influences the motion of the pendulum through mechanical coupling.

The system has two degrees of freedom:

- The rotary arm angle, which is directly actuated.
- The pendulum angle, which is unactuated and influenced by gravity and inertial forces.

The upright position of the pendulum represents an unstable equilibrium point. Stabilization around this position requires continuous feedback control based on real-time measurements of the system states. Angular position sensors are typically used to measure both the arm and pendulum angles, enabling accurate state estimation and implementation of state-space control strategies.

3.1.3. Linear Inverted Pendulum

The linear inverted pendulum system represents an alternative realization of the inverted pendulum problem. In this configuration, the pendulum is mounted on a cart that moves linearly along a horizontal track. The cart motion is actuated, while the pendulum remains unactuated.

Unlike the rotary configuration, the linear inverted pendulum requires a larger physical workspace and is more sensitive to track friction and alignment issues. However, it provides a more intuitive physical interpretation of translational dynamics and is commonly used to introduce inverted pendulum concepts in early control courses.

From a modeling perspective, both systems share similar nonlinear characteristics and unstable equilibrium behavior but differ significantly in their mechanical constraints and actuation mechanisms.

3.1.4. Comparison Between Rotary and Linear Inverted pendulum

Although both systems address the same fundamental control problem—stabilizing an unstable equilibrium they differ in mechanical structure and practical implementation.

The rotary inverted pendulum features a compact and mechanically robust design, making it well suited for laboratory environments with limited space. Its rotational actuation minimizes friction-related issues and improves experimental repeatability.

In contrast, the linear inverted pendulum provides more intuitive visualization of translational motion but requires longer tracks and careful mechanical alignment.

From an educational perspective, the rotary inverted pendulum is particularly suitable for advanced control studies, including state-space modeling, pole placement, linear quadratic regulation, and nonlinear swing-up control, while maintaining close agreement with theoretical models.

3.1.5. Problem statement for Rotary inverted pendulum

Although inverted pendulum systems are extensively covered in control theory courses, they are often studied only through idealized mathematical models and simulations. Such approaches typically neglect practical non-idealities, including actuator limitations, friction, sensor noise, and unmodeled dynamics, which significantly influence real system behavior.

As a result, students may face difficulty translating theoretical control designs into stable and reliable real-time implementations. The rotary inverted pendulum experiment addresses this gap by providing a physical platform on which theoretical models and control strategies can be experimentally validated. It enables direct observation of nonlinear behavior, instability, and under actuation, thereby reinforcing the connection between analytical design and practical control implementation.

3.2. Physical System

3.2.1. Overall Mechanical Architecture

The rotary inverted pendulum system is implemented as a compact bench-scale mechanical platform designed to capture the essential dynamics of an unstable, underactuated system. Overall mechanical architecture is inspired by the educational rotary inverted pendulum developed by Quanser, which serves as a reference benchmark in control engineering education.

The system consists of a rigid base structure supporting a horizontally rotating arm driven about a fixed vertical axis. A pendulum link is mounted at the end of the rotary arm through a low-friction revolute joint, allowing it to rotate freely in the vertical plane. This configuration creates two coupled degrees of freedom, with actuation applied only to the rotary arm.

The pendulum mass and geometry are selected such that the upright position represents an unstable equilibrium. The mechanical components are designed to ensure precise alignment, minimal friction, and sufficient structural rigidity, allowing the experimental behavior to closely match the theoretical dynamic model.

3.2.2. Actuation system and Placement

A compact 12 V DC geared motor designed for smooth, reliable motion in low-speed applications. Its integrated gearbox provides stable torque output and improved control, making it suitable for robotics, automation setups, and educational projects that require durability and consistent performance.

Figure 37. SG555123000 DC Motor

Motor Type	Operating Voltage	No-Load Speed	Rated Current	Stall Torque	Weight
SG555123000. Motor	12 VDC	300 RPM at 12 V	800 mA at 12 V	4 N.cm at 12 V	350 g

Table 5. SG555123000 Motor Specifications

3.2.3. Mechanical Transmission

The motor output shaft is connected to a custom coupling element that serves as a rigid mechanical interface. This coupling is bolted to an L-shaped support bracket, forming a stiff transmission path that securely transfers rotational motion while maintaining precise alignment between the motor and the driven assembly.

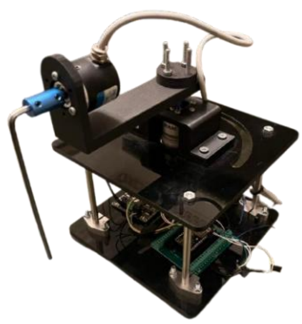


Figure 38. Rotary Inverted Pendulum

3.2.4. Sensors Selection

3.2.4.1. Motor Encoder

The motor is equipped with a **high-resolution incremental encoder with 7 pulses per revolution (PPR)**, providing fine angular resolution over the full rotational range.

The encoder provides two quadrature pulse signals that are processed by the control system to extract the following quantities:

Motor Position: Determined by counting encoder pulses relative to a reference position, providing angular displacement of the motor shaft.

3.2.4.2. Incremental Encoder–Based Pendulum Angle Measurement

The pendulum angular position is measured using a **high-resolution incremental encoder with 2400 pulses per revolution (PPR)**, directly coupled to the pendulum shaft. The encoder generates **two quadrature pulse signals**, which are processed by the control system to determine the pendulum's angular displacement over its full range of motion.

The pendulum angle is obtained by **counting encoder pulses relative to a defined reference position**, enabling precise measurement required for stabilization around the upright equilibrium.

3.2.5. CAD Modeling and Assembly

3.2.5.1.1. Full assembly

The rotary inverted pendulum assembly comprises a rigid base frame supporting a DC motor mounted on the upper plate, a horizontally oriented rotary arm, and a vertically hinged pendulum link attached at the arm's end. The structure is built using stacked plates connected by vertical columns to provide high stiffness and accurate geometric alignment. This compact mechanical arrangement forms a self-contained experimental platform suitable for control system implementation.

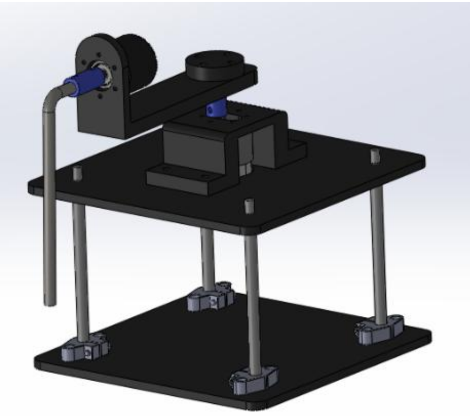


Figure 39. CAD Assembly

3.3. Mathematical Model and System Identification

3.3.1. Dynamic Modeling

The rotary inverted pendulum model considered in this work is illustrated in Figure X. The system consists of a motor-driven rotary arm rigidly coupled to the actuator shaft and rotating in the horizontal plane. The arm has a length L_r, a moment of inertia J_r, and an angular position θ, which is defined to increase positively in the counter-clockwise (CCW) direction. A positive control input results in CCW rotation of the arm.

A rigid pendulum link is mounted at the distal end of the rotary arm and is free to rotate in the vertical plane. The pendulum has a total length L_p, with its center of mass located at a distance $l_p$from the pivot, and a moment of inertia $J_p$about its center of mass. The pendulum angle $\alpha$is defined to be zero when the pendulum is perfectly upright and increases positively for CCW rotation.

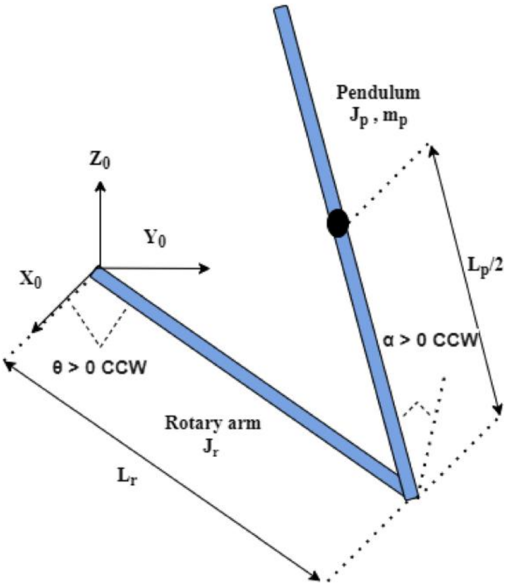


Figure 40. Modeling of Inverted Pendulum

3.3.2. Model Parameters

The model consists of:

- L_r: Length of the rotary arm
- L_p: Total length of the pendulum link
- l_p: Distae from pendulum pivot to its center of mass
- m_p : Mass of the pendulum link
- J_r: Moment of inertia of the rotary arm about the vertical axis
- J_p: Moment of inertia of the pendulum about its center of mass
- B_r: Viscous damping coefficient of the rotary arm joint
- B_p: Viscous damping coefficient of the pendulum joint
- g: Gravitational acceleration

3.3.3. Equations of Motion

The rotary inverted pendulum consists of an actuated rotary arm and a passive pendulum link mounted at the arm tip. The generalized coordinates are chosen as

$$q = [\theta \quad \alpha]^{\top} \tag{3.1}$$

where $\theta$is the rotary arm angle and $\alpha$is the pendulum angle measured from the upright vertical position. Counter-clockwise rotation is defined as positive for both angles

3.3.3.1. Non-Linear Equations of Motion

The nonlinear equations of motion describe the coupled dynamics between the actuated rotary arm and the passive pendulum link. They are derived using the Euler–Lagrange formulation, where the system's kinetic and potential energies are expressed in terms of the generalized coordinates $\theta$and α. The resulting equations capture inertial coupling, gravitational effects, and viscous damping, while the rotary arm is driven by a motor-generated torque. These nonlinear dynamics represent the full physical behavior of the system and form the basis for subsequent linearization about the upright equilibrium used for control design.

$$\begin{aligned}\left(m_pL_r^2 + \frac{1}{4}m_pL_p^2 - \frac{1}{4}m_pL_p^2\cos^2(\alpha) + J_r\right)\ddot{\theta} - \left(\frac{1}{2}m_pL_pL_r\cos(\alpha)\right)\ddot{\alpha} \\ + \left(\frac{1}{2}m_pL_p^2\sin(\alpha)\cos(\alpha)\right)\dot{\theta}\dot{\alpha} + \left(\frac{1}{2}m_pL_pL_r\sin(\alpha)\right)\dot{\alpha}^2 = \tau - B_r\dot{\theta}\end{aligned} \tag{3.2}$$

$$\begin{aligned}-\frac{1}{2}m_pL_pL_r\cos(\alpha)\ddot{\theta} + \left(J_p + \frac{1}{4}m_pL_p^2\right)\ddot{\alpha} \\ -\frac{1}{4}m_pL_p^2\cos(\alpha)\sin(\alpha)\dot{\theta}^2 - \frac{1}{2}m_pL_pg\sin(\alpha) = -B_p\dot{\alpha}\end{aligned} \tag{3.3}$$

3.3.3.2. Motor Torque-voltage Relationship

The motor torque–voltage relationship is incorporated into the rotary arm equation of motion, allowing the system dynamics to be expressed directly in terms of the motor input voltage. This formulation captures the effect of motor back-emf as a velocity-dependent damping term and provides a suitable model for control design.

$$\tau = \frac{\eta_g K_g \eta_m k_t\left(V_m - K_g k_m \dot{\theta}\right)}{R_m} \tag{3.4}$$

The derived equations follow the standard form of equations of motion, where inertial, damping, and gravitational effects are balanced by the applied torque.

$$J\ddot{x} + b\dot{x} + g(x) = \tau_1 \tag{3.5}$$

3.3.4. Linearized Model and State Space representation

3.3.4.1. Upright Equilibrium Linear Model

The nonlinear equations of motion are linearized about the upright equilibrium configuration of the pendulum. This operating point corresponds to zero angular velocities and small deviations of the pendulum angle from the vertical position, allowing the system to be approximated by a linear time-invariant model suitable for control design.

- $\theta = 0$
- $\alpha = 0$
- $\dot{\theta} = 0$
- $\dot{\alpha} = 0$

Under the small-angle assumption, the trigonometric terms are approximated as $\sin(\alpha) \approx \alpha$ and $\cos(\alpha) \approx 1$, while higher-order nonlinear terms are neglected. Applying these approximations to the nonlinear equations of motion yields the following linearized equations, which capture the dominant coupled dynamics of the rotary arm and pendulum near the upright position.

$$M = \begin{bmatrix} J_r + m_p L_r^2 & -\frac{1}{2} m_p L_p L_r \\ -\frac{1}{2} m_p L_p L_r & J_p + \frac{1}{4} m_p L_p^2 \end{bmatrix} \tag{3.6}$$

$$M \begin{bmatrix} \ddot{\theta} \\ \ddot{\alpha} \end{bmatrix} = \begin{bmatrix} \tau - B_r \dot{\theta} \\ \frac{1}{2} m_p L_p \alpha - B_p \dot{\alpha} \end{bmatrix} \tag{3.7}$$

3.3.4.2. Linear State Space Model

The Linear State Space equations are

$$x = Ax + Bu \tag{3.8}$$

And

$$y = Cx + Du \tag{3.9}$$

For the linearized rotary inverted pendulum system, the system matrix $A$and input matrix $B$are obtained directly from the linearized equations of motion. These matrices describe the coupling between the rotary arm and pendulum dynamics, as well as the influence of the motor input voltage on the system states.

$$A = \begin{bmatrix} 0 & 0 & 1 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & a_{32} & a_{33} & a_{34} \\ 0 & a_{42} & a_{43} & a_{44} \end{bmatrix} \tag{3.10}$$

And

$$B = \begin{bmatrix} 0 \\ 0 \\ b_3 \\ b_4 \end{bmatrix} \tag{3.11}$$

Where

$$a_{32} = \frac{\left(\frac{1}{2} {m_p}^2 {L_p}^2 g L_r\right)}{\Delta} \tag{3.12}$$

$$a_{33} = -\frac{\left(J_p + \frac{m_p L_p{}^2}{4}\right) B_r}{\Delta} \tag{3.13}$$

$$a_{34} = \frac{\frac{1}{2} m_p L_p L_r B_p}{\Delta} \tag{3.14}$$

$$a_{42} = \frac{\left(J_r + \frac{m_p L_r{}^2}{4}\right) \frac{1}{2} m_p L_p g}{\Delta} \tag{3.15}$$

$$a_{43} = \frac{\frac{1}{2} m_p L_p L_r B_r}{\Delta} \tag{3.16}$$

$$a_{44} = \frac{\left(J_r + \frac{m_p L_r{}^2}{4}\right) B_p}{\Delta} \tag{3.17}$$

$$b_3 = \frac{J_p + \frac{m_p L_p{}^2}{4}}{\Delta} \tag{3.18}$$

$$b_4 = \frac{\frac{1}{2} m_p L_p L_r}{\Delta} \tag{3.19}$$

Where x is the state, u is the control input, A, B, C, and D are state space matrices. For the rotary pendulum system, the state and output are defined

$$x^T = [\theta \;\; \alpha \;\; \dot{\theta} \;\; \dot{\alpha}] \tag{3.20}$$

And

$$y^T = [x_1 \;\; x_2] \tag{3.21}$$

In the output equation, only the position of the motor and link angles are measured. Based on this, the *C* and *D* matrices in the output equation are

$$C = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix} \tag{3.23}$$

And

$$D = \begin{bmatrix} 0 \\ 0 \end{bmatrix} \tag{3.24}$$

3.3.1. Parameter Estimation

Since the pendulum dynamics are directly influenced by the actuator behavior, the parameter estimation process was structured into **two sequential stages**. First, the electrical and mechanical parameters of the DC motor were identified independently. These identified motor parameters were then fixed and used in the second stage to estimate the remaining mechanical parameters of the rotary arm–pendulum system.

This staged approach improves identifiability, reduces coupling between parameters, and increases confidence in the resulting model.

3.3.1.1. Motor Parameter Estimation

In the first stage, the DC motor driving the rotary arm was modeled independently from the pendulum dynamics. The objective of this phase was to identify the motor's electrical and mechanical parameters without the influence of the pendulum load.

The motor model includes the armature electrical dynamics and the rotational mechanical dynamics, accounting for back electromotive force (EMF), torque generation, inertia, and viscous damping. The applied input to the model is the motor voltage, while the measured output is the motor angular speed.

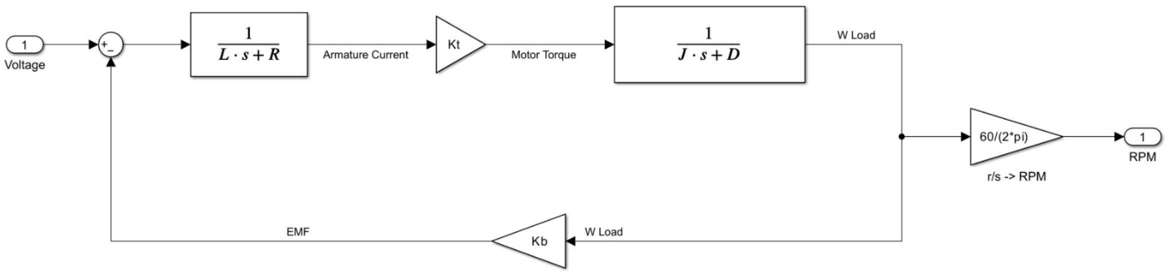


Figure 41. Motor Model

The parameters estimated in this stage include:

- Armature resistance, $R = 3.9336\ \Omega$
- Armature inductance, **L** $= 0.24721$ H
- Motor torque constant, $K_t = 0.39595\ N \cdot m/A$
- Motor inertia, $J = 0.0025202\ kg \cdot m^2$
- Viscous damping coefficient, $D = 4.329 \times 10^{-6}\ N \cdot m \cdot s/rad$

3.3.1.2. System Parameter Estimation

In the second phase, the estimated motor parameters were incorporated into the full rotary inverted pendulum model. The remaining mechanical parameters associated with the rotary arm and pendulum were then identified using the coupled arm–pendulum dynamic equations.

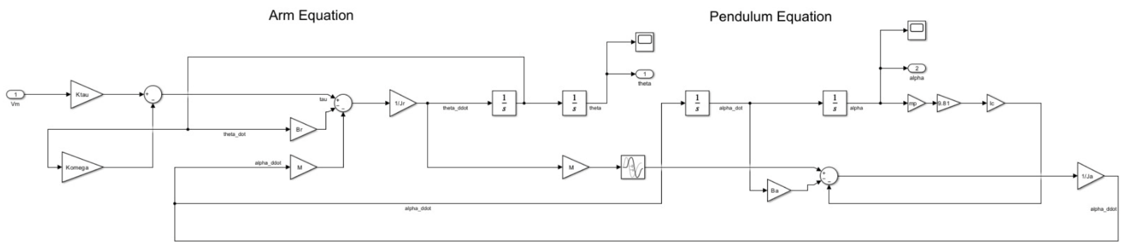


Figure 42. System Linear Model

The parameters estimated in this stage include:

- Rotary arm inertia, $J_r = 0.00061423\ kg \cdot m^2$
- Pendulum inertia, $J_a = 0.00061677\ kg \cdot m^2$
- Rotary arm viscous damping, $B_r = 2.2119 \times 10^{-6}\ N \cdot m \cdot s/rad$
- Pendulum viscous damping, $B_a = 1.1427 \times 10^{-6}\ N \cdot m \cdot s/rad$

3.4. Simulation and Results

3.4.1. Introduction

This chapter presents the simulation framework and performance evaluation of the rotary inverted pendulum system. The primary objective of the simulation is to validate the derived dynamic model, assess the effectiveness of the proposed control strategy, and establish consistency between simulated behavior and experimental results obtained from the physical system.

The simulation environment is developed using Simulink, where the mathematical model of the rotary inverted pendulum is implemented based on the estimated motor and system parameters. The simulation framework supports both open-loop and closed-loop analysis and serves as a foundation for controller design prior to deployment on the physical plant.

3.4.2. Simulation Framework Overview

The simulation framework of the rotary inverted pendulum system is developed using an integrated **SolidWorks–Simulink–Simscape Multibody** workflow to ensure strong consistency between the mechanical design and the dynamic model. The CAD model of the rotary arm and pendulum assembly is first created in SolidWorks and then imported into Simulink using Simscape Multibody, where it forms the basis of the physical plant representation.

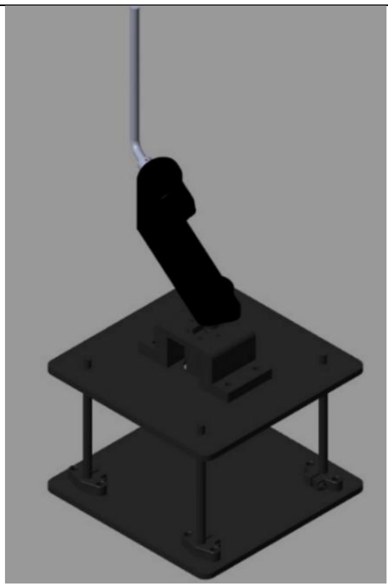


Figure 43. Sim-mechanics Simulation

Simscape Multibody is used to model the rigid-body dynamics, joints, inertial properties, and kinematic constraints of the pendulum system, while Simulink is employed to implement the control logic, reference inputs, signal conditioning, and performance evaluation. This integrated approach allows the mechanical structure, physical dynamics, and control system of the rotary inverted pendulum to be developed and evaluated within a unified simulation environment that closely reflects the real experimental setup.

3.4.3. Overall Simulink Model Architecture

The overall Simulink model integrates the Simscape Multibody–based plant, controller subsystem, reference generation, and monitoring blocks into a closed-loop simulation framework. At this level, the focus is on system-level signal flow and interaction between subsystems rather than internal physical modeling details.

The model is structured to allow independent development and testing of the plant and controller. Measured states from the plant are routed to the controller, which computes the motor input command. System outputs are logged and visualized to evaluate stabilization performance and dynamic response.

3.4.3.1. Hardware In the Loop Model (HIL)

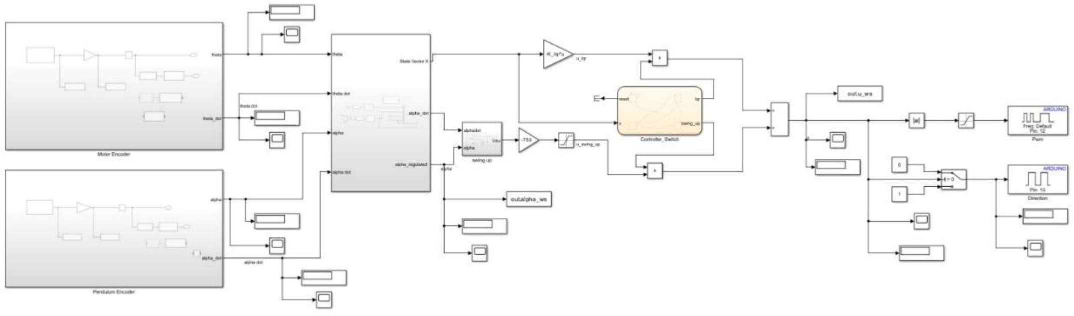


Figure 43. HIL

3.4.3.2. Model In the Loop Model (MIL)

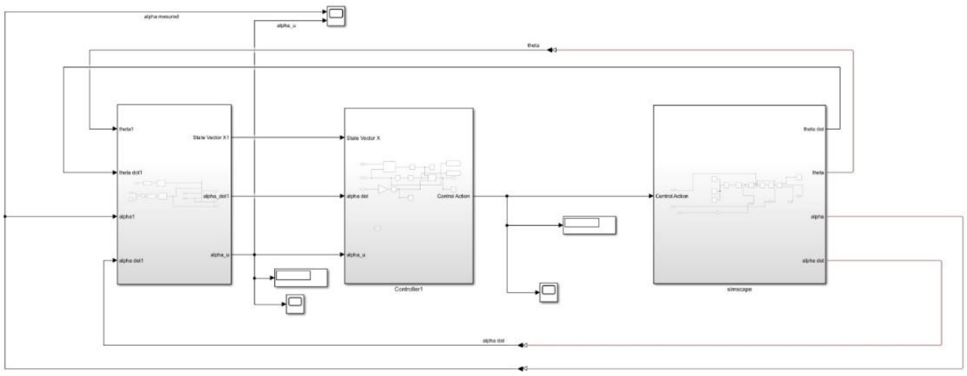


Figure 44. MIL

3.4.4. Plant Subsystem Description

The plant subsystem represents the complete physical model of the rotary inverted pendulum and is implemented using Simscape Multibody components imported from the SolidWorks CAD model. It captures the rigid-body dynamics of the rotary arm and pendulum, including inertial properties, joint constraints, and gravitational effects, while incorporating the estimated motor and mechanical parameters obtained from the parameter estimation stage to ensure accurate representation of the physical system.

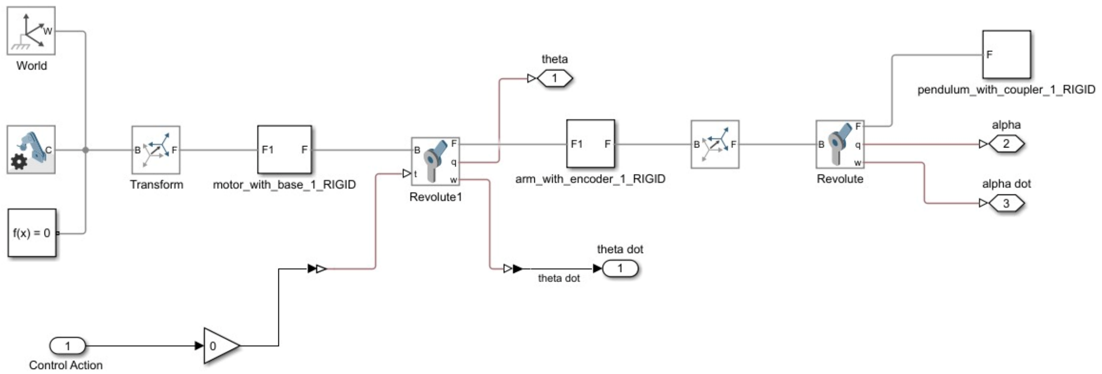


Figure 45. Plant Subsystem

3.4.5. Controllers' Subsystem Description

The controller subsystem is responsible for driving the pendulum to the upright position and maintaining its stabilization. Two control modes are implemented within Simulink: a swing-up controller and an LQR stabilizing controller. The swing-up controller is initially active and is used to inject energy into the system to bring the pendulum close to the upright configuration. Once the pendulum angle αenters a predefined region around the upright equilibrium, control is smoothly transferred to the LQR controller, which takes over to stabilize the pendulum and regulate the rotary arm motion. This hierarchical control structure ensures reliable transition from large-angle motion to precise stabilization while maintaining a modular design that allows each control strategy to be modified or extended independently.

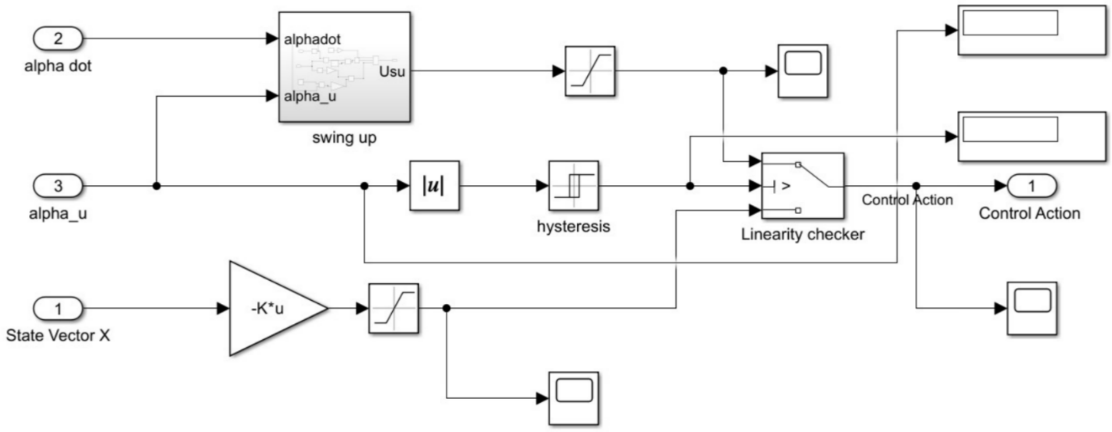


Figure 46. Controller Subsystem

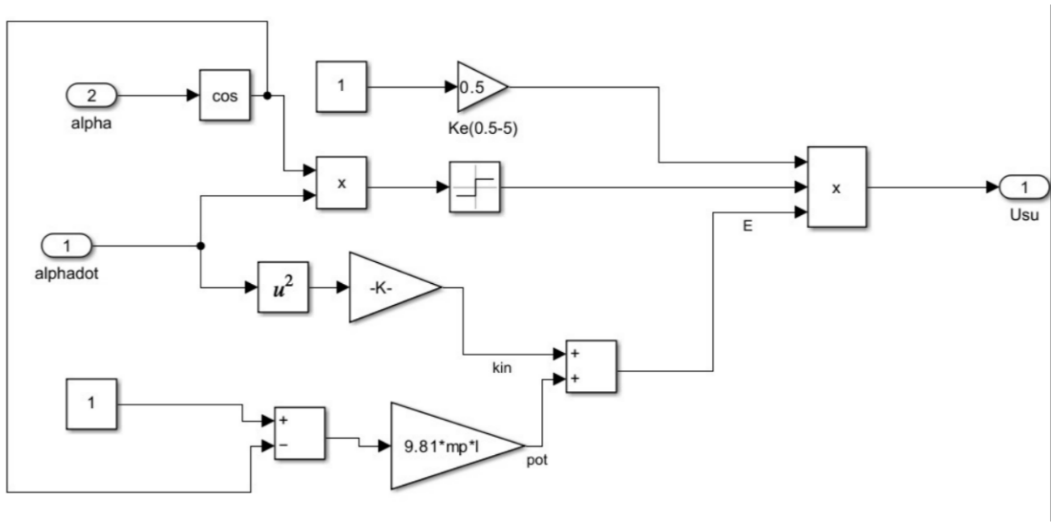
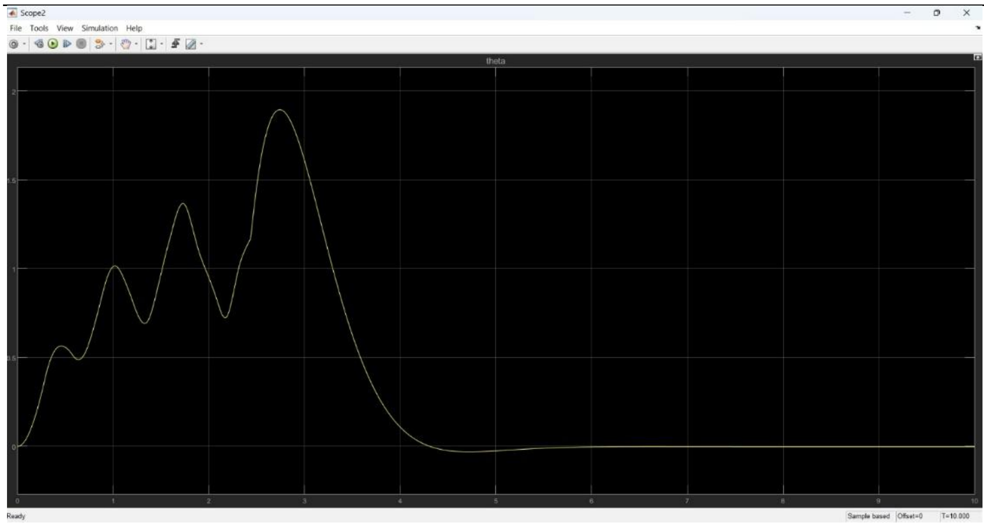
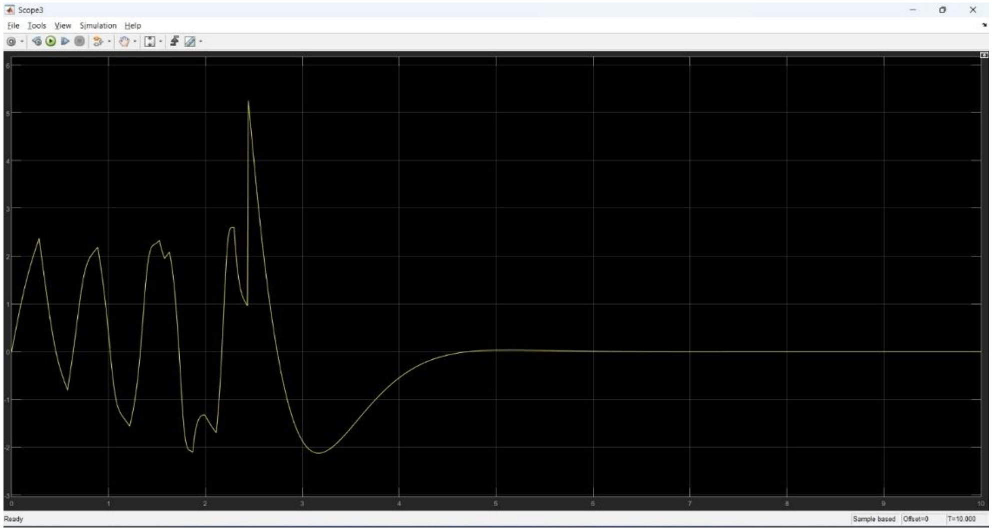


Figure 47. Swing up control

3.4.6. Simulation and Experimental Results

The performance of the rotary inverted pendulum system is evaluated through both simulation and real-time implementation on the physical plant. The system response is assessed using the pendulum angle α, pendulum angular velocity $\dot{\alpha}$, rotary arm angle θ, and rotary arm angular velocity $\dot{\theta}$. The results show successful swing-up and stabilization of the pendulum around the upright equilibrium, with close qualitative agreement between simulated and experimental responses, confirming the validity of the dynamic model and the effectiveness of the implemented control strategy.


Scope2
File Tools View Simulation Help
theta
0 1 2 3 4 5 6 7 8 9 10
Ready
Sample based
Offset=0
T=10.000


Scope3
File Tools View Simulation Help
0 1 2 3 4 5 6 7 8 9 10
Ready
Sample based
Offset=0
T=10.000

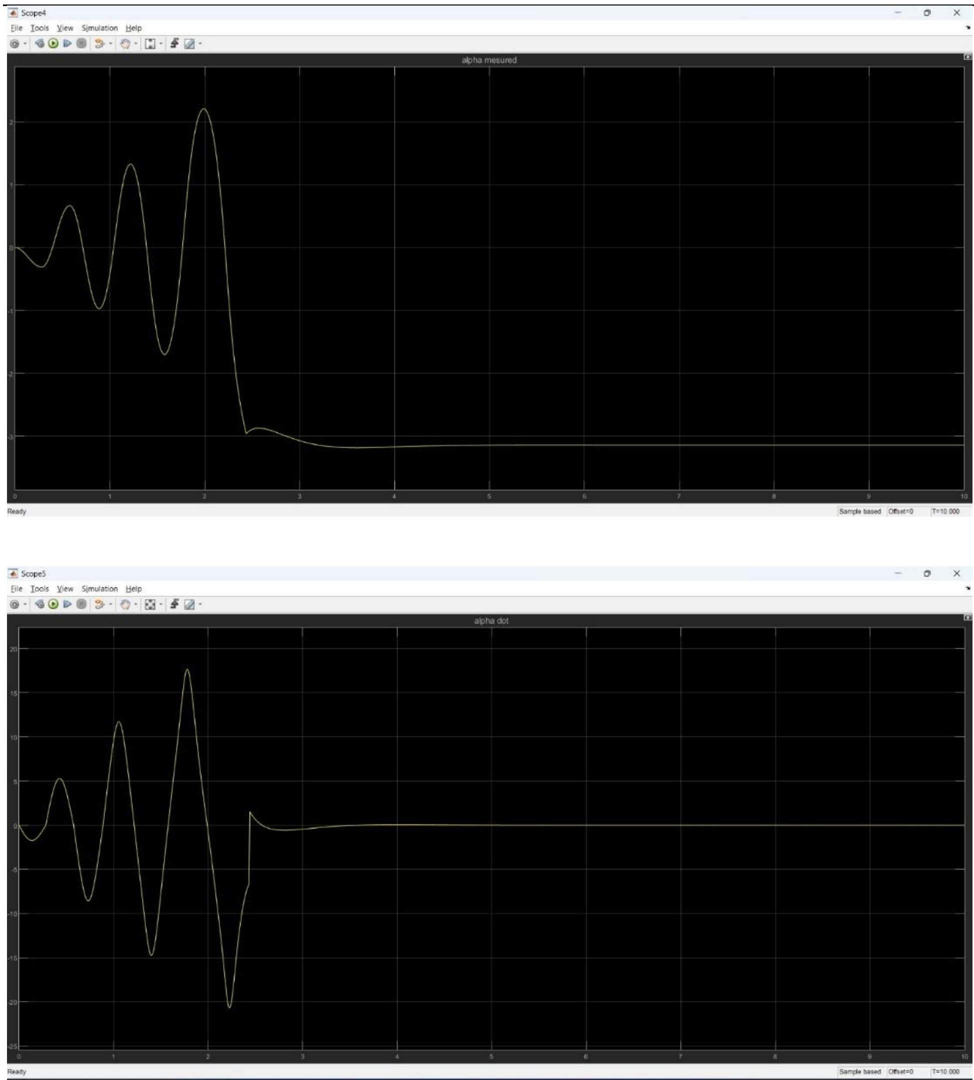


Figure 48. Simulation Results

4. Conclusion

This work presented the design, modeling, control, and evaluation of two control-oriented experimental platforms: an active suspension system and a rotary inverted pendulum. The primary objective was to develop modular, physically realistic systems that support both simulation-based analysis and practical control experimentation within an educational laboratory environment.

For the active suspension system, a complete mechanical and electrical architecture was developed and integrated into a physics-based simulation framework using a SolidWorks–Simulink–Simscape workflow. An LQR controller was designed and evaluated through simulation under representative road excitation. The results demonstrated clear performance improvements with active control compared to passive operation, particularly in terms of reduced suspension travel, improved road handling, and enhanced ride comfort. These results validate the effectiveness of the proposed control strategy and establish readiness for subsequent hardware implementation.

For the rotary inverted pendulum, both the dynamic model and control strategy were implemented and validated in simulation and on the actual physical system. The controller was successfully deployed on the real plant, demonstrating stable real-time operation and confirming the consistency between the simulated model and the experimental behavior. This dual validation highlights the robustness of the modeling approach and the suitability of the pendulum setup as a hands-on platform for studying unstable systems and real-time control.

Overall, this work establishes a coherent framework that bridges mechanical design, electrical integration, simulation, and control implementation. The developed platforms provide a solid foundation for advanced control experimentation, future extensions, and integration into a remote-access laboratory environment.

5. Practical Challenges

During the design, integration, and testing of the experimental platforms, several practical challenges were encountered. These challenges are inherent to real-world mechatronic systems and provided valuable insights into the gap between idealized models and physical implementation. This section outlines the main issues observed and the mitigation strategies adopted or planned.

5.1. Sensor Noise and Measurement Uncertainty in Active Suspension Sensors

Observed Issue

- High noise levels were observed in sensor measurements, even when the system was in a static equilibrium state.
- This behavior was present in both displacement (ToF) and inertial (IMU) sensors.

Impact on the System

- Noisy and unreliable sensor readings, even at equilibrium, which reduced confidence in the measured states.
- Inconsistent feedback signals, making the measurements unsuitable for direct use in closed-loop control.
- Delay in deploying control on the physical plant, as additional state estimation and filtering were required before safe and reliable control implementation.

Current Handling

- Basic low-pass filtering applied in Simulink to attenuate high-frequency noise.

Planned Mitigation (Next Development Phase)

- Implementation of a Kalman filter–based state estimation framework.
- Sensor fusion to optimally combine noisy measurements with system dynamics.

5.2. Capstan Mechanism Tension Loss

Observed Issue:

During rapid downward motion of the suspension system, the capstan string became loose.

This resulted in partial loss of force transmission from the actuator.

Root Cause:

- Slippage between the string and the capstan pulley due to insufficient mechanical engagement.

Impact on the System

- Non-ideal actuator behavior.
- Denying oscillations of the top plate in the open loop mode

Implemented Solution

- A mechanical modification was made by adding a guiding hole in the capstan pulley.
- The string is routed through the hole before fixation to the moving plate, ensuring positive engagement.
- This design change successfully eliminated slippage and restored consistent force transmission.

5.3. Low Encoder Resolution in the Rotary Pendulum

Observed Issue:

- The DC motor driving the rotary arm is equipped with an incremental encoder with a very low resolution of 7 pulses per revolution (PPR).

Root Cause:

- The encoder resolution corresponds to an angular resolution of approximately **3°** per count, which is insufficient for precise angular feedback in an inverted pendulum system.

Impact on the System:

- Coarse and quantized measurement of the rotary arm angle.
- Inability to accurately detect small angular deviations around the upright equilibrium position.

Planned Mitigation (Next Development Phase)

- Replacing the encoder with a higher-resolution device.

6. Future Work and Second Semester Roadmap

The work presented in this thesis establishes a solid foundation for both experimental platforms. The following activities are planned for the second semester to extend the system capabilities, transition from simulation to real-time implementation, and enhance the educational value of the laboratory. The roadmap is organized according to technical dependency and implementation priority.

6.1. Sensor Fusion and State Estimation Enhancement

Objective

- Improve measurement reliability and state estimation accuracy for Active Suspension sensors

Planned Work

- Design and implement a Kalman filter–based state estimation framework.
- Fuse displacement and inertial sensor measurements with system dynamics.

Timeline

- Weeks 1–2 of the second semester

6.2. Real-Time Control of Deployment on the Active Suspension System

Objective

- Apply the previously designed control strategies to the actual active suspension hardware.

Planned Work

- Deploy control algorithms on the physical plant after sensor noise mitigation.
- Evaluate real-time performance and compare experimental results with simulation.

Dependency

- Completion of sensor fusion and state estimation enhancement.

Timeline

- Week 2 of the second semester

6.3. Cloud Networked Model Predictive Control (CNMPC)

Objective

- Extend the control framework by introducing a cloud-executed MPC controller.

Planned Work

- Develop an MPC formulation for the active suspension system.
- Execute control optimization remotely via a the CoTune cloud interface to align with the laboratory vision of remote experimentation

Timeline

- Week 6 of the second semester

6.4. Digital Twin Development

Objective

- Create digital twins for both the active suspension system and the rotary inverted pendulum.

Planned Work

- Synchronize real-time experimental data with simulation models.
- Enable live monitoring and system visualization.
- Support remote experimentation and student interaction.

Timeline

- Week 10 of the second semester

6.5. Reinforcement Learning (RL) Tutor Module

Objective

- Introduce learning-based control as an educational extension to classical methods.

Planned Work

- Develop an RL-based control tutor for guided experimentation.
- Allow students to compare RL performance with LQR and MPC controllers.
- Integrate the module as an optional advanced learning tool.

Timeline

- Week 12 of the second semester

7. References

[1] S. L. Brunton and J. N. Kutz, "Linear Control Theory," in *Data-Driven Science and Engineering: Machine Learning, Dynamical Systems, and Control*, Cambridge University Press, Cambridge, UK, pp. 276–320, 2019.

[2] K. Ogata, *Modern Control Engineering*, 5th ed. Upper Saddle River, NJ, USA: Prentice Hall, 2009.

[3] K. J. Åström and R. M. Murray, *Feedback Systems: An Introduction for Scientists and Engineers*. Princeton, NJ, USA: Princeton University Press, 2008.

[4] C.-T. Chen, *Linear System Theory and Design*, 3rd ed. Oxford, UK: Oxford University Press, 1998.

[5] N. S. Nise, *Control Systems Engineering*, 6th ed. Hoboken, NJ, USA: John Wiley & Sons, 2008.

[6] B. Friedland, *Control System Design: An Introduction to State-Space Methods*. New York, NY, USA: McGraw-Hill, 1986.

[7] R. Rajamani, *Vehicle Dynamics and Control*, 2nd ed. New York, NY, USA: Springer, 2012.

[8] MathWorks, "Simscape Multibody User's Guide," MathWorks Inc., 2024. [Online]. Available: https://www.mathworks.com

[9] Quanser Inc., "Active Suspension Experiment – Student Workbook," Quanser Educational Resources. [Online]. Available: https://www.quanser.com

8. Appendices

MATLAB Codes

Active Suspension LQR Script

```
%%  ACTIVE SUSPENSION CONTROL
% Purpose: Initialize Simscape Data, Build Plant, and Design LQR Controller
clear; clc; close all;

%% 1. INITIALIZATION & DATA LOADING
% Load Simscape mechanical data (Transforms, Masses, etc.)
addpath ActiveSuspension
run(fullfile('ActiveSuspension', 'ACTIVESUSPENSIONSYS_OPT_DataFile.m'))

%% 2. PHYSICAL PARAMETERS (Quanser Nominal)
% Values confirmed from your main and hardware manual
Ms  = 1.42;     % Sprung mass [kg]
Mus = .580;     % Unsprung mass [kg]
Ks  = 697;      % Suspension stiffness [N/m]
Kus = 1140;     % Tire stiffness [N/m]
Bs  = 5.0;      % Suspension damping [N*s/m]
Bus = 3.6;      % Tire damping [N*s/m]

%% 3. STATE-SPACE REPRESENTATION (Force Input)
% States: [z_s-z_us; dz_s; z_us-z_r; dz_us]
A = [ 0       1         0          -1;
     -Ks/Ms   -Bs/Ms     0          Bs/Ms;
      0        0         0           1;
      Ks/Mus   Bs/Mus   -Kus/Mus   -(Bs+Bus)/Mus ];

% Control input u = Fc (Active Force in Newtons)
```

```
B = [0; 1/Ms; 0; -1/Mus];

%% 4. SYSTEM ANALYSIS (Professional Checks)
% Controllability Check: Must be rank 4
Co = ctrb(A,B);
fprintf('Controllability rank = %d/4\n', rank(Co));

%% 5. LQR DESIGN (Tuning)
% Q matrix: Penalty on states [Travel, Velocity, Deflection, Tire_Vel]
% High values on Q(1,1) and Q(3,3) prioritize stability and ride height.
Q = diag([800, 200, 800, 20]);

% R matrix: Penalty on Force (N). Smaller R = more aggressive control.
R = 0.05;

% Calculate Optimal Gain K
[K, S, E] = lqr(A, B, Q, R);

fprintf('\n--------------------------------------------------\n');
fprintf('LQR Gain K calculated for FORCE input (N):\n');
fprintf('K = [%.4f  %.4f  %.4f  %.4f]\n', K(1), K(2), K(3), K(4));
fprintf('Closed-loop poles:\n');
disp(E);
fprintf('--------------------------------------------------\n');

%% 6. LAUNCH SIMULINK
open(fullfile('ActiveSuspension', 'ACTIVESUSPENSIONSYS.slx'))
fprintf('Simulink model is ready. Set external Gain to 1 to activate LQR.\n');
```

Rotary Inverted Pendulum LQR Script

```
%% Rotary Inverted Pendulum (Upright) - A, B (Vm), and LQR Gain
% State: x = [theta; alpha; theta_dot; alpha_dot]
```

```
% Conventions: CCW positive for theta and alpha, alpha = 0 at upright.

clc; clear; close all;

%% Motor Parameters
Rm = 3.9336;        % Ohm
kt = 0.39595;       % N*m/A
km = 0.39595;       % V/(rad/s)
Kg = 1;             % Gear ratio
eta_g = 1;          % Gear efficiency (already inside kt per your note)
eta_m = 1;          % Motor efficiency

%% Physical Parameters
mp = 0.02;          % kg
Lp = 0.134;         % m (total length)
Lr = 0.185;         % m
Jp = 2.9927e-5;     % kg*m^2 (about CM)
Jr = 2.852e-3;      % kg*m^2
Br = 0.003;         % N*m*s/rad
Bp = 0.00005;       % N*m*s/rad
g  = 9.81;          % m/s^2

%% ===== Linearized model about upright (alpha ~ 0) with TORQUE input =====
% From workbook EOM (linearize sin(alpha)~alpha, cos(alpha)~1, drop products)
% M * [theta_ddot; alpha_ddot] = [ tau - Br*theta_dot ;
%                                  +0.5*mp*Lp*g*alpha - Bp*alpha_dot ]
%
% where:
Jr_t = Jr + mp*Lr^2;            % effective arm inertia term at alpha=0
Jp_t = Jp + 0.25*mp*Lp^2;          % effective pendulum inertia term
Jx   = 0.5*mp*Lp*Lr;             % coupling term
```

```
detM = Jr_t*Jp_t - (Jx^2);

% A,B for torque input u = tau
A_tau = zeros(4,4);
B_tau = zeros(4,1);

A_tau(1,3) = 1;
A_tau(2,4) = 1;

% theta_ddot = (1/detM)*( Jp_t*(tau - Br*theta_dot) + Jx*(0.5*mp*Lp*g*alpha - Bp*alpha_dot) )
A_tau(3,2) = (Jx * (0.5*mp*Lp*g)) / detM;   % alpha term
A_tau(3,3) = -(Jp_t * Br) / detM;           % theta_dot term
A_tau(3,4) = -(Jx   * Bp) / detM;           % alpha_dot term
B_tau(3)   = (Jp_t) / detM;                 % tau term

% alpha_ddot = (1/detM)*( Jx*(tau - Br*theta_dot) + Jr_t*(0.5*mp*Lp*g*alpha - Bp*alpha_dot) )
A_tau(4,2) = (Jr_t * (0.5*mp*Lp*g)) / detM; % alpha term
A_tau(4,3) = -(Jx   * Br) / detM;           % theta_dot term
A_tau(4,4) = -(Jr_t * Bp) / detM;           % alpha_dot term
B_tau(4)   = (Jx) / detM;                   % tau term

%% ===== Convert TORQUE input to VOLTAGE input Vm (workbook Eq. 2.4 style) =====
% tau = (eta_g*Kg*eta_m*kt/Rm)*Vm - (eta_g*Kg^2*eta_m*kt*km/Rm)*theta_dot
KtVm = (eta_g*Kg*eta_m*kt)/Rm;           % tau per Vm
Kbemf = (eta_g*(Kg^2)*eta_m*kt*km)/Rm;   % tau per theta_dot (back-emf term)

A = A_tau;
B = B_tau;

% Add back-emf term into A via the same pattern used in workbook snippet:
```

```
A(3,3) = A(3,3) - Kbemf * B(3);
A(4,3) = A(4,3) - Kbemf * B(4);

% Scale B to convert input from tau to Vm:
B = KtVm * B;

% Output matrices (if you need them)
C = [1 0 0 0;
     0 1 0 0];
D = [0;0];

%% Display A, B
disp('A (Vm input) ='); disp(A);
disp('B (Vm input) ='); disp(B);

%% ===== LQR design =====
% Tune Q and R as you like (start here, then adjust).
% Q = diag([  1000,  5000,   2,  50 ]);   % [theta, alpha, theta_dot, alpha_dot]
% R = 0.5;                              % voltage effort penalty

% Q = diag([  800,  2100,   2,  50 ]);   % [theta, alpha, theta_dot, alpha_dot]
% R = 0.25;



%%%%%%%% BEST SO FAR %%%%%%%%---------------------------------------------

% Q = diag([  2000,  8000,   50,  50 ]);   % [theta, alpha, theta_dot, alpha_dot]
% R = 1;

% Q = diag([  2000,  8000,   50,  75 ]);   % [theta, alpha, theta_dot, alpha_dot]
% R = 1;
```

```
% Q = diag([  2000,  8000,   50,  100 ]);   % [theta, alpha, theta_dot, alpha_dot]
% R = 1;

% Q = diag([  2000,  12000,   50,  75 ]);   % [theta, alpha, theta_dot, alpha_dot]
% R = 1;

%------------------------------------------------------------------------
Q = diag([  2000,  12000,   50,  75 ]);   % [theta, alpha, theta_dot, alpha_dot]
R = 1;
% MATLAB returns K_lqr for u = -K_lqr*x
K_lqr = lqr(A,B,Q,R);

% If you implement u = K*(xd - x) (Quanser style), use:
K_use = -K_lqr;

disp('K_lqr (for u = -K_lqr*x) ='); disp(K_lqr);
disp('K_use (for u =  K_use*(xd - x)) ='); disp(K_use);

% Quick sign check (you wanted [+ - + -] on K_use):
disp('sign(K_use) ='); disp(sign(K_use));
```

S-Function Block Code

```
/* Includes_BEGIN */
#ifndef MATLAB_MEX_FILE
#include <Arduino.h>
#include <Wire.h>
#include "Adafruit_VL53L0X.h"

// -------------------- Addresses --------------------
#define LOX1_ADDRESS 0x30
#define LOX2_ADDRESS 0x31
#define MPU1_ADDRESS 0x68   // AD0=GND
```

```
#define MPU2_ADDRESS 0x69   // AD0=3.3V

// -------------------- Shutdown Pins ----------------
#define SHT_LOX1 25
#define SHT_LOX2 26

// -------------------- I2C pins ---------------------
#define I2C_SDA 21
#define I2C_SCL 22

// -------------------- Objects ----------------------
static Adafruit_VL53L0X lox1;
static Adafruit_VL53L0X lox2;

static VL53L0X_RangingMeasurementData_t measure1;
static VL53L0X_RangingMeasurementData_t measure2;

// -------------------- Shared data (ToF) ------------
static volatile int32_t g_dist1_mm = -1;
static volatile int32_t g_dist2_mm = -1;
static volatile uint32_t g_tof_t_us = 0;     // timestamp of last ToF update (micros)
static volatile bool g_lox1_ok = false;
static volatile bool g_lox2_ok = false;

// Critical section for shared variables (not I2C!)
static portMUX_TYPE g_dataMux = portMUX_INITIALIZER_UNLOCKED;

// -------------------- I2C Mutex (THE IMPORTANT FIX)-
static SemaphoreHandle_t g_i2cMutex = NULL;

// -------------------- Helpers ----------------------
static inline bool I2C_Lock(TickType_t timeoutTicks)
```

```
{
  if (!g_i2cMutex) return true; // fallback (shouldn't happen)
  return (xSemaphoreTake(g_i2cMutex, timeoutTicks) == pdTRUE);
}

static inline void I2C_Unlock()
{
  if (g_i2cMutex) xSemaphoreGive(g_i2cMutex);
}

static inline void i2cWriteByte(uint8_t addr, uint8_t reg, uint8_t val)
{
  Wire.beginTransmission(addr);
  Wire.write(reg);
  Wire.write(val);
  Wire.endTransmission(true);
}

static inline bool i2cReadBytes(uint8_t addr, uint8_t startReg, uint8_t *buf, size_t len)
{
  Wire.beginTransmission(addr);
  Wire.write(startReg);
  if (Wire.endTransmission(false) != 0) return false; // repeated start
  int got = Wire.requestFrom((int)addr, (int)len, (int)true);
  if (got != (int)len) return false;
  for (size_t i = 0; i < len; i++) buf[i] = Wire.read();
  return true;
}

static inline void wakeUpMPU(uint8_t address)
{
  // PWR_MGMT_1 = 0 -> wake
```

```
  i2cWriteByte(address, 0x6B, 0x00);
}

static inline void configMPU6050(uint8_t address)
{
  // Baseline config for control/estimation:
  // DLPF ~ 42 Hz, Fs = 200 Hz, Accel ±4g, Gyro ±500 dps.
  wakeUpMPU(address);
  delay(2);

  // CONFIG (0x1A) DLPF_CFG=3 -> ~42 Hz
  i2cWriteByte(address, 0x1A, 0x03);

  // SMPLRT_DIV (0x19) = 4 -> 1000/(1+4)=200 Hz (when DLPF enabled)
  i2cWriteByte(address, 0x19, 0x04);

  // ACCEL_CONFIG (0x1C) AFS_SEL=1 -> ±4g
  i2cWriteByte(address, 0x1C, 0x08);

  // GYRO_CONFIG (0x1B) FS_SEL=1 -> ±500 dps
  i2cWriteByte(address, 0x1B, 0x08);
}

static inline bool readMPUAccelXYZ(uint8_t address, int16_t &ax, int16_t &ay, int16_t &az)
{
  uint8_t buf[6];
  if (!i2cReadBytes(address, 0x3B, buf, 6)) return false;
  ax = (int16_t)((buf[0] << 8) | buf[1]);
  ay = (int16_t)((buf[2] << 8) | buf[3]);
  az = (int16_t)((buf[4] << 8) | buf[5]);
  return true;
```

```
}

// ------------- VL53L0X address assignment -----------
static void setVL53IDs()
{
  // Reset both
  digitalWrite(SHT_LOX1, LOW);
  digitalWrite(SHT_LOX2, LOW);
  delay(10);

  // Bring both high
  digitalWrite(SHT_LOX1, HIGH);
  digitalWrite(SHT_LOX2, HIGH);
  delay(10);

  // Keep #1 awake, #2 shut down
  digitalWrite(SHT_LOX1, HIGH);
  digitalWrite(SHT_LOX2, LOW);
  delay(10);

  // IMPORTANT: protect I2C during begin()
  if (I2C_Lock(pdMS_TO_TICKS(50))) {
    g_lox1_ok = lox1.begin(LOX1_ADDRESS, false); // false: don't restart Wire
    I2C_Unlock();
  } else {
    g_lox1_ok = false;
  }
  delay(10);

  // Now wake #2
  digitalWrite(SHT_LOX2, HIGH);
  delay(10);
```

```
  if (I2C_Lock(pdMS_TO_TICKS(50))) {
    g_lox2_ok = lox2.begin(LOX2_ADDRESS, false);
    I2C_Unlock();
  } else {
    g_lox2_ok = false;
  }
  delay(10);
}

// ------------ ToF background task ------------------
static void ToFTask(void *param)
{
  // Run ToF at a realistic rate. 20 ms -> 50 Hz.
  const TickType_t period = pdMS_TO_TICKS(20);
  TickType_t lastWake = xTaskGetTickCount();

  for (;;)
  {
    vTaskDelayUntil(&lastWake, period);

    int32_t d1 = -1, d2 = -1;

    // Lock the I2C bus for the whole ToF transaction
    if (!I2C_Lock(pdMS_TO_TICKS(30))) {
      // Couldn't get the bus quickly; skip this cycle (do not block control loop)
      continue;
    }

    if (g_lox1_ok) {
      lox1.rangingTest(&measure1, false);
      if (measure1.RangeStatus != 4) d1 = (int32_t)measure1.RangeMilliMeter;
```

```
    }
    if (g_lox2_ok) {
      lox2.rangingTest(&measure2, false);
      if (measure2.RangeStatus != 4) d2 = (int32_t)measure2.RangeMilliMeter;
    }

    I2C_Unlock();

    portENTER_CRITICAL(&g_dataMux);
    g_dist1_mm = d1;
    g_dist2_mm = d2;
    g_tof_t_us = micros();
    portEXIT_CRITICAL(&g_dataMux);
  }
}
#endif
/* Includes_END */

/* Externs_BEGIN */
/* extern double func(double a); */
/* Externs_END */

void Sensors_Start_wrapper(void)
{
/* Start_BEGIN */
#ifndef MATLAB_MEX_FILE
  pinMode(SHT_LOX1, OUTPUT);
  pinMode(SHT_LOX2, OUTPUT);

  // Create I2C mutex FIRST
  g_i2cMutex = xSemaphoreCreateMutex();
```

```
  // I2C init once
  Wire.begin(I2C_SDA, I2C_SCL);
  Wire.setClock(400000);      // 400 kHz
  delay(20);

  // Configure MPUs (protected)
  if (I2C_Lock(pdMS_TO_TICKS(50))) {
    configMPU6050(MPU1_ADDRESS);
    delay(5);
    configMPU6050(MPU2_ADDRESS);
    I2C_Unlock();
  }
  delay(5);

  // Setup ToFs with unique addresses
  setVL53IDs();

  // Initialize shared values
  portENTER_CRITICAL(&g_dataMux);
  g_dist1_mm = -1;
  g_dist2_mm = -1;
  g_tof_t_us = micros();
  portEXIT_CRITICAL(&g_dataMux);

  // Start ToF task (core 0)
  xTaskCreatePinnedToCore(ToFTask, "ToFTask", 4096, NULL, 1, NULL, 0);
#endif
/* Start_END */
}

void Sensors_Outputs_wrapper(int32_T *Dist1,
                             int32_T *Dist2,
```

```
                    int16_T *MPU1,
                    int16_T *MPU2)
{
/* Output_BEGIN */
#ifndef MATLAB_MEX_FILE
  // -------- 1) Read MPUs at Ts (your synchronized tick) --------
  int16_t ax1=0, ay1=0, az1=0;
  int16_t ax2=0, ay2=0, az2=0;

  bool ok1 = false, ok2 = false;

  // Lock I2C briefly for MPU reads
  if (I2C_Lock(pdMS_TO_TICKS(5))) {
    ok1 = readMPUAccelXYZ(MPU1_ADDRESS, ax1, ay1, az1);
    ok2 = readMPUAccelXYZ(MPU2_ADDRESS, ax2, ay2, az2);
    I2C_Unlock();
  }

  if (!ok1) {
    // Sentinel for "bad read"
    ax1 = ay1 = az1 = (int16_t)0x8000;
    // Try to re-wake (protected)
    if (I2C_Lock(pdMS_TO_TICKS(5))) { wakeUpMPU(MPU1_ADDRESS);
I2C_Unlock(); }
  }
  if (!ok2) {
    ax2 = ay2 = az2 = (int16_t)0x8000;
    if (I2C_Lock(pdMS_TO_TICKS(5))) { wakeUpMPU(MPU2_ADDRESS);
I2C_Unlock(); }
  }

  // NOTE: MPU1 and MPU2 ports MUST be dimensioned [3x1] in S-Function Builder
  MPU1[0] = ax1; MPU1[1] = ay1; MPU1[2] = az1;
```

```
  MPU2[0] = ax2; MPU2[1] = ay2; MPU2[2] = az2;

  // -------- 2) Sample-and-hold latest ToF (no blocking) --------
  int32_t d1, d2;
  portENTER_CRITICAL(&g_dataMux);
  d1 = g_dist1_mm;
  d2 = g_dist2_mm;
  portEXIT_CRITICAL(&g_dataMux);

  Dist1[0] = d1;
  Dist2[0] = d2;

#endif
/* Output_END */
}

void Sensors_Terminate_wrapper(void)
{
/* Terminate_BEGIN */
/*
 * Custom Terminate code goes here.
 */
/* Terminate_END */
}
```